Oldenburgische
Industrie- und Handelskammer



[1]

Abschlussprüfung Sommer 2015

Fachinformatiker Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

Erweiterung einer Webanwendung um Task Notifications





Prüfungsbewerber:

Björn Nowak
Lamberti Str. 72
26121 Oldenburg
Geb.: 10.07.1993 in Bremen
Azubi-Ident.-Nr.: 0001014762

Ausbildungsbetrieb:

BTC AG
Escherweg 5
26121 Oldenburg

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

[1] http://www.hfci.de/uploads/images/zoom/ihk-oldenburg-logo.png

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Vorwort . . . 1
1.2 Kurzporträt der Ausbildungsstätte . . . 1
1.3 Projektumfeld . . . 1
1.4 Motivation und Zielsetzung . . . 2
1.5 Erklärung zum Dokument . . . 2
1.6 Aufbau des Dokuments . . . 2

2 Planungsphase **2**

2.1 Zeitlicher Rahmen des Projektes . . . 2
2.2 Vorgehensweise . . . 2
2.3 Verwendete Hardware . . . 3
2.4 Verwendete Software/Technologien . . . 3

3 Analysephase **4**

3.1 Ist-Analyse . . . 4
3.2 Soll-Konzept . . . 4
3.3 Zukünftig mögliche Anforderungen . . . 5
3.4 Wirtschaftlichkeitsbetrachtung . . . 5

4 Konzeptphase **7**

4.1 Technisches Konzept . . . 7
4.2 UI-Entwurf . . . 7
4.3 Proof-of-Concept . . . 8
4.4 Testplan . . . 8

5 Realisierungsphase **9**

5.1 Projektstruktur . . . 9
5.2 Einrichten/Konfigurieren von Atmosphere . . . 9
5.3 Kommunikation: Server zu Client . . . 9
5.4 Datenhaltung . . . 10
5.5 Datenübertragung . . . 11
5.6 Event-Handling . . . 12
5.7 UI-Entwurfsumsetzung . . . 12

6 Qualitätssicherungsphase **13**

6.1 Definition of Done . . . 13
6.2 Testergebnisse . . . 13

7 Abschlussphase **13**

7.1 Soll-Ist-Vergleich . . . 13
7.2 Persönliches Fazit . . . 14

Literaturverzeichnis i

A Anhang ii

A.1 Weboberfläche . . . ii
A.1.1 UI-Entwurf . . . ii
A.1.2 listOpenTasks.xhtml (Auszug) . . . ii
A.1.3 main.less (Auszug) . . . iii
A.1.4 masterTemplate.xhtml (Auszug) . . . v
A.2 Tabellen . . . vi
A.2.1 Laptop-Spezifikation . . . vi
A.2.2 Auszug der vorhandene Paketstruktur . . . vi
A.3 Quellcode . . . vii
A.3.1 AbstractNotificationService.java . . . vii
A.3.2 AtmosphereConfig.java . . . viii
A.3.3 main.js (Auszug) . . . viii
A.3.4 NotificationController.java . . . ix
A.3.5 NotificationServiceHandler.java . . . x
A.3.6 OpenTaskListController.java . . . xii
A.3.7 pom.xml (Auszug) . . . xiii
A.3.8 PostRenderView-Event . . . xiv
A.3.9 TaskNotificationEvent.java . . . xiv
A.3.10 TaskNotificationService.java . . . xv
A.3.11 TaskQueryServiceImpl.java (Auszug) . . . xvi
A.3.12 UserStatus(Repository).java . . . xviii
A.3.13 VmsWebAppInitializer.java (Auszug) . . . xx
A.4 Weitere Grafiken . . . xxi
A.4.1 UML-Klassendiagramm (Übersicht) . . . xxi
A.4.2 UML-Klassendiagramm (Komplex) . . . xxii

Tabellenverzeichnis

2 Testfälle 8
3 Testergebnisse 13
4 Ursprüungliche Zeitplanung und tatsächlich benötigte Zeit im Vergleich 14
5 Laptop-Spezifikation vi
6 Packetstruktur: Fachliche Saeulen vi
7 Packetstruktur: Technische Layer vi

Abbildungsverzeichnis

1 Systemlandschaft 1
2 Mockup Status-Anzeige - Einbettung und Verknüpfung zur Task-Übersicht 5
3 Differenz zur vorherigen Packetstruktur 9
4 UI-Entwurf ii
5 UML-Klassendiagramm (Übersicht) xxi
6 UML-Klassendiagramm (Komplex) xxii

Abkürzungsverzeichnis

API Application Programming Interface

BTC Business Technology Consulting AG

CSS Cascading Style Sheets

HTML Hypertext Markup Language

HTML DOM *HTML* Document Object Model

IDE Integrated Development Environment

Java EE Java Enterprise Edition

JSON JavaScript Object Notation

SQL Structured Query Language

TSO Transmission System Operator

TSO BSS TSO Business Service Suit

UI User Interface

VMS Vertragsmanagementsystem

XML Extensible Markup Language

1 Einleitung

1.1 Vorwort

Im Jahre 2012 begann ich meine Ausbildung zum Fachinformatiker für Anwendungsentwicklung bei der *Business Technology Consulting AG (BTC)*. Zuvor habe ich eine schulische Ausbildung zum staatlich geprüfter TAI[2] absolviert und erlangte an derselben Schule die Fachhochschulreife. Seit Mitte Juli 2013 arbeite ich, fast durchgängig, in dem im Kapitel 1.3 beschriebenem Team.

Es liegt die Dokumentation des Projektes „Erweiterung einer Webanwendung um Task Notifications“ vor. Dieses Projekt habe ich im Rahmen der angestrebten IHK-Ausbildung bei der *BTC* durchgeführt. Es stellt einen möglichen Prozess der Erweiterung einer bestehenden Webanwendung dar.

1.2 Kurzporträt der Ausbildungsstätte

Die BTC Business Technology Consulting AG ist eines der führenden IT-Consulting-Unternehmen in Deutschland mit Niederlassungen in der Schweiz, der Türkei, Polen und Japan. Das Unternehmen mit mehr als 1.600 Mitarbeitern und Hauptsitz in Oldenburg ist Partner von SAP und Microsoft. *BTC* erzielte in 2013 einen Umsatz von 195,2 Mio. Euro.

BTC hat ein ganzheitliches, auf Branchen ausgerichtetes IT-Beratungsangebot und damit eine führende Position in den Bereichen Energie, Telekommunikation, Öffentlicher Sektor, Industrie und Dienstleistungen. Das Angebot umfasst Beratung, Systemintegration sowie Applikations- und Systemmanagement. Ein weiterer Fokus liegt auf energienahen Softwareprodukten.

Seit der Gründung im Jahre 2000 hat die *BTC* ein dichtes Netz an Geschäftsstellen aufgebaut, um IT-Consulting kundennah erbringen zu können (Oldenburg, Berlin, Bremen, Hamburg, Leipzig, Mainz, Münster, Neckarsulm, Zürich, Poznań, Istanbul, Ankara, Tokio). [BTC 2014]

1.3 Projektumfeld

Das Projekt war im Entwicklerteam I des Bereiches „Non-SAP Technologien“ der „Business Unit Markets“ angesiedelt. Dieses Team ist hauptsächlich für die Entwicklung des Kundenprojektes *TSO Business Service Suit (TSO BSS)* zuständig. Daraus resultiert unter Anderem die Webanwendung *Vertragsmanagementsystem (VMS)*. Das Projekt ist als Teilprojekt der allgemeinen Weiterentwicklung des *VMS* zu sehen. Im Folgenden gehe ich daher kurz auf die *TSO BSS* und das *VMS* ein.

Die *TSO BSS* gehört dem Sektor Energie an und ist ein Vertrags- und Kapazitätsmanagementsystem mit einer engen Anbindung zu Vermarktungsplattformen. Die Webanwendung *VMS* dient der Verwaltung von Kundenverträgen. Zusätzlich unterstützt das System sowohl in fachlichen als auch technischen Prozessen. Als Kommunikationszentrale spricht das *VMS* mit nachgelagerten Systemen des *Transmission System Operator (TSO)* sowie mit externen Systemen, wie z. B. Vermarktungsplattformen (Siehe Abbildung 1) [vgl. BTC 2015].

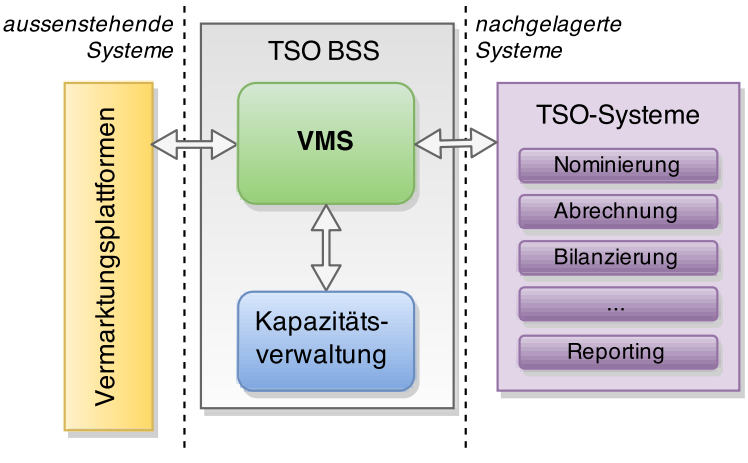


Abbildung 1: Systemlandschaft

[2] Technischer Assistent für Informatik.

1.4 Motivation und Zielsetzung

BTC hat für das *VMS* eine Erweiterung identifiziert und dem Kunden vorgestellt. Der Kunde hat dann die Umsetzung beauftragt.

Der Benutzer soll zu jeder Zeit auf einen Blick die Task-Anzahl, der ihm oder seiner Benutzerrolle zugewiesenen offenen Tasks, sehen können. Hinsichtlich der Benutzerfreundlichkeit sollte diese Anzeige so in Form und Farbe gestaltet werden, dass diese intuitiv und auch für Menschen mit Sehschwächen, wie Farbenblindheit, ohne Einschränkungen nutzbar ist [Björn Nowak 2015].

1.5 Erklärung zum Dokument

An dieser Stelle wird die Gestaltung des Dokuments erläutert.

- Ein [*kursiver Text*] weist auf eine Abkürzung hin, die im Abkürzungsverzeichnis zu finden ist.
- Ein [**fettgedruckter Text**], mit Ausnahme von Überschriften, weist auf einen Fachbegriff hin, zu dem im Dokument eine Erklärung vorliegt. Ausgenommen sind Erklärungen in Fußnoten.
- Ein [`Text in Schreibmaschinenschrift`] weist auf eine Klasse oder Methode hin.
- Ein [Text in serifloser Schrift] weist auf sonstige Programmcode-Textbausteine hin.
- Liegt das Dokument in elektronischer Form vor, sind Abkürzungen und Quellenangaben zum jeweiligen Verzeichniseintrag verlinkt.

1.6 Aufbau des Dokuments

Einführend wird ein Überblick über den Aufbau der vorliegenden Projektdokumentation gegeben.

Im folgenden Kapitel wird auf die Rahmenbedingungen und Vorgehensweiße zur Durchführung des Projektes eingegangen. Im dritten Kapitel wird die zugrundeliegende Webanwendung analysiert, Anforderungen an das Projekt definiert und die Wirtschaftlichkeit dargelegt. Die festgelegten Anforderungen werden im Kapitel Vier zu einem Konzept zusammengefasst. Das anschließende fünfte Kapitel erklärt wie das Konzept umgesetzt wurde. Nachfolgend wird im sechsten Kapitel aufgezeigt wie die Softwarequalität bewahrt wurde. Abschließend wird die Projektdurchführung in Kapitel Sieben zusammengefasst betrachtet und ein persönliches Fazit gezogen.

2 Planungsphase

2.1 Zeitlicher Rahmen des Projektes

Für die Durchführung der betrieblichen Projektarbeit steht ein Zeitrahmen von 70 Arbeitsstunden zur Verfügung. Der Durchführungszeitraum beginnt am 09. März 2015 und endet am 19. März 2015. Um diesen Zeitraum nicht zu überschreiten ist es erforderlich, die im Projektantrag festgelegten Meilensteine sowohl zeitlich als auch inhaltlich einzuhalten. Das Risiko durch abweichende Anforderungen wird mit einem einberechneten Puffer begrenzt.

2.2 Vorgehensweise

Zur Umsetzung des Projektes wird eine iterativ inkrementelle Vorgehensweise gewählt. Bei diesem Vorgehen wird das Gesamtprojekt schrittweise entwickelt, also in Teilprojekte (oder auch Iterationen) aufgeteilt. Jedes dieser Teilprojekte durchlebt dabei alle Entwicklungsdisziplinen, von der Analyse bis zur Implementierung. Nach jeder Iteration kann die neu gewonnene Erfahrung und möglicherweise erhaltenes Feedback, unmittelbar in die nächste Iteration einfließen. Darauffolgende Iterationen können auch zur kontinuierlichen Verbesserung vorheriger Teilprojekte dienen.

Diese zyklische Entwicklung soll durch das Einbinden neuster Erkenntnisse, z.B. Anforderungensänderungen, eine hohe Softwarequalität sichern und die Einhaltung der Anforderungen gewährleisten.

2.3 Verwendete Hardware

Neben dem in der Tabelle 5: Laptop-Spezifikation auf S. vi aufgeführten Laptop wurden eine Dockingstation, mit einer 1:1 Portreplikation, und zwei 24" Bildschirme (HP Compaq LA2405x) genutzt.

2.4 Verwendete Software/Technologien

Die nachfolgende Liste ist aus Gründen der Übersichtlichkeit auf einen Auszug der wichtigsten Spezifikationen reduziert worden.

Intellij IDEA Ist eine kostenpflichtige, dafür sehr mächtige und umfangreiche, integrierte Entwicklungsumgebung (von engl.: *Integrated Development Environment (IDE)*).

Java EE Die Plattform *Java Enterprise Edition (Java EE)*, ehemals J2EE ist eine umfangreiche Sammlung von Softwarekomponenten und Diensten. Ferner werden Spezifikationen, Code Standards und Bibliotheken definiert. Mit der Java EE Plattform wurde ein Rahmen für Softwareentwickler geschaffen, in dem verteilte, transaktions-basierte und mehrschichtige Unternehmensanwendungen entwickelt werden können. Klar definierte Bibliotheken und Schnittstellen / *Application Programming Interface (API)*'s zwischen den Java EE Komponenten sorgen dafür, dass auch Software-Komponenten von unterschiedlichen Herstellern zusammenarbeiten können (z.B. SAP Anbindung). [UWS Ltd 2015]

Atmosphere Das Open-Source-Framework Atmosphere ermöglicht es, in *Java EE* skalierbare und asynchrone Echtzeit-Anwendungen zu entwickeln. Dabei unterstützt das Framework eine Reihe von Applikationsservern und versteckt zugleich deren komplexe Servlet[3] *API* der Version 3.0. Ein besonderes Merkmal ist, dass das Framework je nach Applikationsserver und dem aufrufendem Browser die geeignete Transportmethode auswählen kann.

Spring Das Spring Framework (kurz Spring) ist ein quelloffenes Framework für die Java-Plattform. Ziel des Spring Frameworks ist es, die Entwicklung mit Java/*Java EE* zu vereinfachen und gute Programmierpraktiken zu fördern. Spring bietet mit einem breiten Spektrum an Funktionalität eine ganzheitliche Lösung zur Entwicklung von Anwendungen und deren Geschäftslogiken; dabei steht die Entkopplung der Applikationskomponenten im Vordergrund. [Wikipedia 2015]

jQuery Bei jQuery handelt es sich um eine plattformunabhängige und umfangreiche Klassenbibliothek für die Scriptsprache JavaScript. Diese Open-Source-Software vereinfacht den Zugriff und die Manipulation des *HTML Document Object Model (HTML DOM)* und wird insbesondere für ihre performante Programmierung wertgeschätzt. [vgl. ITwissen.info 2015]

LESS Mit LESS wird *Cascading Style Sheets (CSS)* um dynamisches Verhalten erweitert. Damit kann die Wartbarkeit sowie Übersichtlichkeit von *CSS* gesteigert werden. So sind z. B. Vererbungen und die Definierung von Variablen mit unterschiedlichen Geltungsbereichen und Funktionen möglich. Es entspricht einer Annäherung zum objekt-orientierten Programmierparadigma.

[3] Eine Java-Klasse die HTTP Request's annimmt und mit einem entsprechenden Response beantworten kann.

3 Analysephase

Ziel der Analysephase ist es, Anforderungen zu definieren, die bis zum Projektabschluss in der bestehenden *Java EE*-Anwendung umgesetzt werden müssen. Dazu wird in der Ist-Analyse das bisherige System hinsichtlich der Task-Übersicht untersucht und beschrieben. Im Anschluss schildert das Soll-Konzept, wie die Zielsetzung[4] mit dem Entwicklungsstand zum Zeitpunkt der Betrachtung erreicht werden soll. Nach einer kurzen Prüfung weiterer zukünftiger Anforderungen wird abschließend eine für das Projektumfeld hinreichende Wirtschaftlichkeitsbetrachtung dargelegt.

3.1 Ist-Analyse

Ein Task wird vom *VMS* automatisch generiert und ermöglicht bei bestimmten Ereignissen die Interaktion mit den Benutzern. Diese Task sind in vordefinierte Task-Typen unterteilt: Fehler, Information und Benutzerinteraktion (z.B. Bestätigung einer Vertragsänderung oder eines neuen Benutzers).

In der Task-Übersicht findet der Endanwender alle offenen Tasks sowie die in Bearbeitung befindendlichen Task des eigenen Benutzers und seiner Benutzergruppen. Ein Benutzer kann nur aktiv nachsehen wie viele Tasks oder Info-Meldungen existieren. Dazu muss er die Task-Übersicht öffnen und nach einem spezifischen Task-Typ filtern. Erst dann wird die Anzahl der Task-Einträge für diesen Task-Typ ersichtlich. Der Benutzer wird weder auf neue Tasks hingewiesen, noch augenblicklich über diese informiert. [vgl. Björn Nowak 2015]

3.2 Soll-Konzept

Um einen schnellen Zugriff auf die Informationen „Anzahl der offenen Tasks pro Typ“ gewährleisten zu können, sollen diese in die Navigationsleiste neben dem Menüpunkt „Tasks“ integriert werden. Die Darstellung erfolgt getrennt nach Task-Typ. Als Darstellungsmittel werden einfache Symbole genutzt, welche die genannte Anzahl beinhalten. Dabei erhält jeder Typ eine Form und eine Farbe. Als Begrifflichkeit dafür wird im weiterem Verlauf „Status-Anzeige“ genutzt. Dies kann sich auf die Anzeige eines oder aller Task-Typs beziehen.
Dem Kunden sind die Task-Typen „Fehler“ und „Benutzerinteraktion“ besonders wichtig. Die Status-Anzeige der Tasks soll sich daher auf diese beiden beschränken.
Der Fachanwender kann außerdem visuell explizit auf neue Tasks hingewiesen werden. Des Weiteren können die jeweiligen Status-Anzeigen auf die Task-Übersicht verlinkt werden. Die Übersicht wird dann entsprechend des Task-Typs gefiltert angezeigt. Dabei darf der Benutzer-spezifische Filter in der Weboberfläche visuell nicht verändert werden und muss weiterhin mit dem Benutzer-bekannten „Filter anwenden“-Button nutzbar sein. Die Abbildung 2: Mockup Status-Anzeige - Einbettung und Verknüpfung zur Task-Übersicht auf S. 5 dient zur Veranschaulichung dieser Anforderung.

Zur Optimierung der Netzwerk- und Systemlast, muss darauf geachtet werden, dass die Anzeige vom *VMS* nur dann aktualisiert wird, wenn sich die angezeigte Information serverseitig geändert hat. Das herkömmliche Verfahren des sog. „polling“ ist ungeeignet, da jeder Benutzer in einem festgelegtem Intervall die Informationen abfragt und damit eine erhöhte Last auf dem System erzeugt. Daher soll unter Anderem auf bekannte Verfahren wie „Comet“ oder auch „long-polling“ gesetzt werden. Bei diesen wird eine Anfrage zunächst vom Server nicht beantwortet sondern suspendiert. Liegt serverseitig ein Event vor, wird diese angehaltene Verbindung entsprechend beantwortet. Erst dann stellt der Browser erneut eine Anfrage. Er erhält also nur eine Antwort, wenn der Server dazu Bedarf sieht [vgl. Joe Hanson 2014].

[4] Siehe Kapitel 1.4: Motivation und Zielsetzung auf Seite 2.

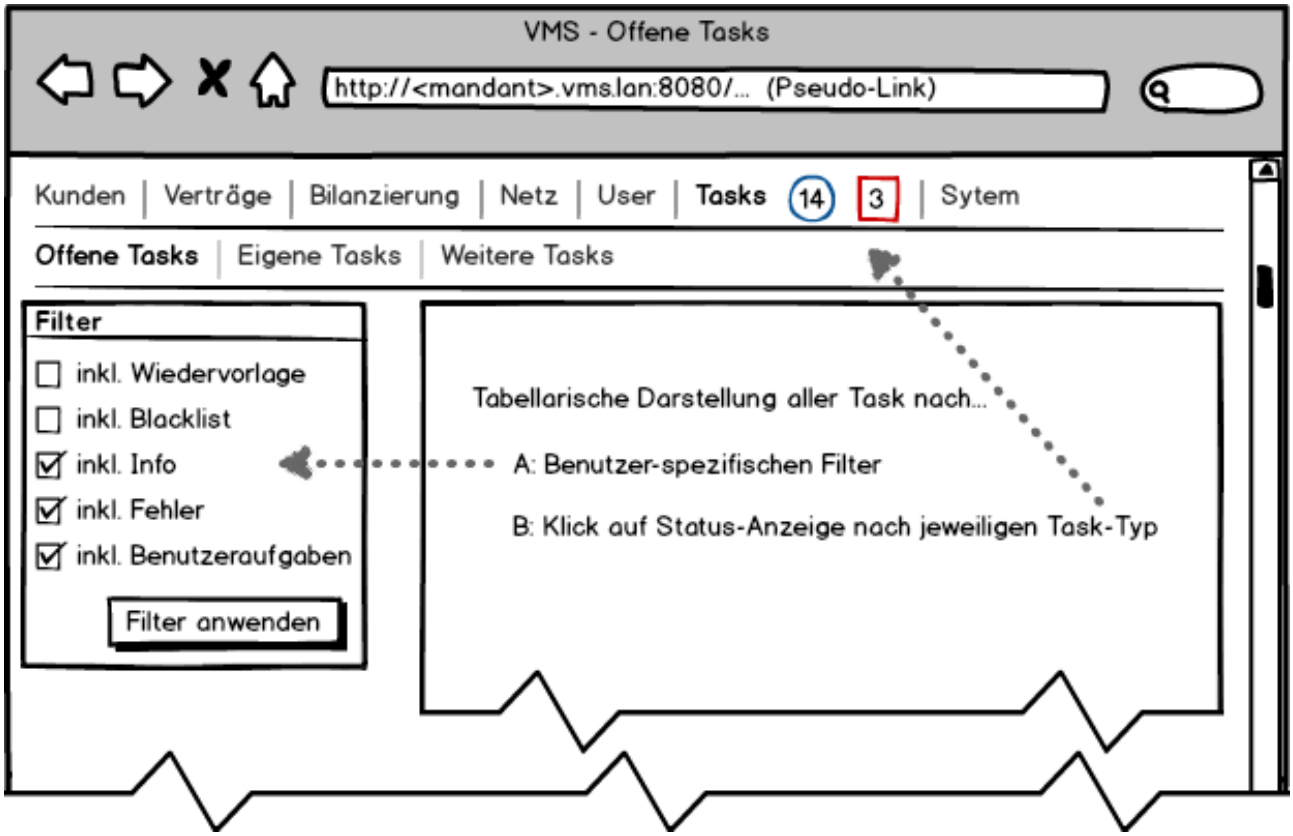


Abbildung 2: Mockup Status-Anzeige - Einbettung und Verknüpfung zur Task-Übersicht

3.3 Zukünftig mögliche Anforderungen

Da es sich bei der *TSO BSS* um ein Langzeitprojekt handelt, werden zukünftig zu erwartende Anforderungen im Konzept berücksichtigt.

Denkbar ist beispielsweise, dass die hier eingeführte Event-Technologie auch für andere Zwecke verwendet wird. Es könnte z. B. gefordert werden, dass eine bestimmte Benutzergruppe direkten Einfluss auf einen Hintergrundprozess nehmen kann. Ein Prozess teilt dem *VMS* mit, dass eine Benutzerinteraktion erforderlich ist. Daraufhin löst *VMS* ein Event aus, dass bei einem Benutzer ein Popup anzeigt, welches die Aktion nachfragt.
Des Weiteren könnte sich der Kunde dahingehend umentscheiden, dass nicht nur die in Kapitel 3.2 genannten Task-Typen angezeigt werden sollen, sondern auch weitere.
Als Letztes wird davon ausgegangen, dass neben dem letzten Besuch der Offenen-Task-Übersicht eines Benutzers in Zukunft weitere Benutzer-abhängige Daten im *VMS* gesichert werden sollen. Dies wurde jedoch erst im Laufe der Realisierungsphase spezifiziert und im Rahmen des Puffers für Anforderungsänderungen, in Kapitel 5.4.1: UserStatusRepository auf Seite 11, beachtet.

3.4 Wirtschaftlichkeitsbetrachtung

Zur Einhaltung des ökonomischen Prinzips wird die Wirtschaftlichkeit des Projektes überprüft. Dazu werden im Folgenden zunächst die zu ermittelten Projektkosten mit dem erwarteten Nutzen verglichen. Abgeschlossen wird diese Betrachtung mit einer Make-or-Buy-Entscheidung, die darlegt, warum nur eine Eigenentwicklung möglich ist und zur Realisierung ein kostenfreies Framework genutzt wird.

3.4.1 Kosten-Analyse

Die Kosten für das Projekt berechnen sich aus den Entwicklungskosten sowie den Kosten für zukünftige Anpassungen und Erweiterungen des daraus entstandenen Features.

Zur Berechnung der Entwicklungskosten wird die in Kapitel 2.1 auf Seite 2 genannte Projektdauer von 70 Stunden genutzt und ein interner Verrechnungssatz von 45 € pro Stunde[5] verwendet. Dieser setzt sich aus verschiedenen Kostenfaktoren zusammen. Dazu zählen unter anderem: Lohn- und Lohnnebenkosten, Energie, IT-Infrastruktur, Verwaltung, Nutzung von Räumlichkeiten etc. Somit ergibt sich für die reinen Entwicklungskosten ein Wert in Höhe von:

$$70h * 45\frac{€}{h} = 3150\,€$$

3.4.2 Nutzen-Analyse

In der Nutzen-Analyse wird aufgezeigt, welcher Nutzen sich für die *BTC* aus der Investition in das Projekt „Erweiterung einer Webanwendung um Task Notifications“ ergibt. Dabei muss zwischen monetärem und nicht-monetärem Nutzen unterschieden werden.

3.4.2.1 Monetärer Vorteil Ein monetärer Vorteil entsteht im Regelfall durch die Verkaufserlöse einer Software. In Fall von *VMS* handelt es sich um eine Individualsoftware für einen Kunden, die kontinuierlich im Auftrag des Kunden weiterentwickelt wird. Der monetäre Vorteil erschließt sich also durch das Anbieten und gewinnbringende Verkaufen dieser Erweiterung in der nächsten Beauftragung. Der Erlös wird mit einem Verrechnungssatz von 145 € pro Stunde[6] ermittelt. Daraus kann der zu erwartene Gewinn wie folgt berechnet werden:

$$\left(70h * 145\frac{€}{h}\right) - 3150\,€ = 7000\,€$$

3.4.2.2 Nicht-monetärer Vorteil Ein nicht-monetärer Aspekt liegt vor, wenn dieser gar nicht bis unzureichend monetär bewertbar ist. Durch das Anbieten der Erweiterung signalisiert die *BTC* gegenüber dem Kunden das Verständnis seiner Bedürfnisse. Dies führt zu einer Verbesserung der Kundenzufriedenheit und -beziehung, welches wiederum zu weiteren Aufträgen führen kann. Das Task-Benachrichtigungssystem führt auch zu einer Verbesserung des operativen Tagesgeschäfts beim Kunden. So können die Mitarbeiter schneller über bestimmte Ereignisse informiert werden, was eine Optimierung der Reaktionszeiten bedeutet. Sobald in Oktober 2015 in der Energie-Branche stündliche Vermarktungen eingeführt werden, wird beim Kunden dieser Faktor an Gewicht gewinnen.

3.4.3 Make-or-Buy-Entscheidung

Die in Kapitel 3.2: Soll-Konzept auf Seite 4 beschriebene Erweiterung ist so individuell, dass am Markt keine Lösung angeboten wird. Selbst im Falle eines standarisierten Marktproduktes könnte dieses nicht ohne größere Anpassungen und Implementierungsarbeiten eingesetzt werden. Da es sich außerdem bei *VMS* um eine ganzheitliche Eigenentwicklung der *BTC* handelt, soll auch weiterhin auf eine betriebsinterne Entwicklung gesetzt werden.

[5] Ausgedachter Wert; Der interne Satz darf nicht bekannt gegeben werden.
[6] Ausgedachter Wert; Der reelle Satz darf nicht bekannt gegeben werden.

3.4.3.1 Framework Die Umsetzung der Kommunikation zwischen Server und Client sowie der Verbindungshaltung stellt einen kritischen Faktor dar. Das Atmosphere Framework bietet dafür geeignete Verfahren gemäß Kapitel 3.2: Soll-Konzept auf Seite 4 an. Auch um im Rahmen des magischen Dreiecks[7]: „Zeit, Qualität und Kosten“ zu bleiben, wird auf das frei erhältliche Framework gesetzt. Dieses reduziert die benötigte Entwicklungszeit so, dass die geplante Projektdauer ohne Einschnitte in der Produktqualität eingehalten werden kann.

4 Konzeptphase

In dieser Phase soll, vorbereitend für die Realisierungsphase, ein Konzept zur Umsetzung der bereits beschrieben Anforderungen aufgezeigt werden. Inklusive jener aus Kapitel 3.3: Zukünftig mögliche Anforderungen auf Seite 5. Dies beinhaltet ein technisches Konzept, ein *User Interface (UI)*-Entwurf sowie die Aufstellung eines Testplans.

4.1 Technisches Konzept

Die Anforderungen werden in technische Arbeitspakete gruppiert, wie in Kapitel 2.2: Vorgehensweise auf Seite 2 erläutert. Daraus resultieren die folgenden Arbeitspakete: Konfiguration von Atmosphere, Kommunikation: Server zu Client, Datenhaltung, Datenübertragung, Event-Handling und *UI*-Entwurfsumsetzung. Im Folgenden werden einige Pakete auszugsweise genauer spezifiziert.

Kommunikation: Server zu Client Das Framework **Atmosphere** bietet weitreichende Features an, daher sollen bei der Umsetzung möglichst viele Aufgaben vom Framework übernommen werden. Wie z. B. der Verbindungsaufbau nach einem unerwarteten Abbruch und ein Nachrichten-Cache, um Nachrichten neu zu versenden, die ein angemeldeter Client nicht erhalten hat. Ein angemeldeter Client bedeutet in diesem Kontext, ein Browser der eine bestehende Verbindung zum Server hat. Auch das Senden von sog. „Heartbeats“ (zu Deutsch: Herzschläge) kann konfiguriert werden. Diese werden benötigt um die Verbindung bei zwischengeschalteten Proxies, welche lang suspendierte Verbindungen unterbrechen, aufrecht zu erhalten.

Datenhaltung In einem Repository (Siehe Kapitel 5.4.1 auf Seite 11) sollen Benutzer-abhängige Informationen mit niedriger Kritikalität gehalten werden. Eine niedrige Kritikalität liegt vor, wenn es akzeptabel ist, die Information z. B. durch einen Serverneustart zu verlieren. Diese wurde zuvor in der Datenbank abgelegt.

Datenübertragung Daten vom Server zum Client/Browser werden im Format *JavaScript Object Notation (JSON)*[8] übermittelt. Dazu soll jede Event-Klasse eine generalisierte Klasse erweitern. Diese bietet eine entsprechende Methode an, welche die Event-Daten im *JSON*-Format zurückliefert. Damit wird sichergestellt, dass übertragende Daten immer in diesem Format vorliegen. Es ist den abgeleiteten Klassen erlaubt, diese Methode zu überschreiben, um den Inhaltsumfang selbst zu definieren.

4.2 UI-Entwurf

Die grafische Benutzeroberfläche ist ein zentraler Bestandteil des *VMS*, weil sie die Schnittstelle zwischen Anwender und Software ist. Da bereits eine Weboberfläche vorhanden ist, müssen Änderungen in diese integriert werden. Der Gestaltungsspielraum wird dadurch stark eingeschränkt: Es ist daher verstärkt darauf zu achten, dass die Änderungen gut mit der restlichen Oberfläche harmonisieren. Da diese größtenteils aus Graustufen besteht werden ausschließlich Pastelltöne verwendet.

[7] Vgl.: http://www.projektmanagementhandbuch.de/projektinitiierung/zieldefinition/ (Abruf: 24.03.2015).
[8] Erklärung siehe [SELFHTML e.V. 2015].

Die Umsetzung der *UI* erfolgte prototypisch und ist im Anhang A.1.1: UI-Entwurf auf S. ii zu sehen. Die Idee aus Kapitel 3.2 auf Seite 4 dem Benutzer auf explizit neue Tasks hinzuweisen, wurde dabei bereits aufgenommen. Es wurde vernachlässigt unterschiedliche Formen pro Task-Typ zu entwerfen.

4.3 Proof-of-Concept

Es wurde eine Demo-Implementierung mit dem Framework Atmosphere vorgenommen. Dazu wurde ein Chat-Beispiel nachgebaut. Das Augenmerk lag dabei auf der Einrichtung von Atmosphere, sowie der richtigen Konfiguration der Webapplikation für eine asynchrone Verarbeitung. Weil keine Minderung der Serverperformance festgestellt wurde, konnte mit dem Framework fortgefahren werden.

4.4 Testplan

In einem Testplan werden die definierten Anforderungen zusammengefasst und beschrieben wie diese zu prüfen sind. Dies sichert bereits vor der Realisierungsphase die vollständige Umsetzung aller Anforderungen. Neben den in der Tabelle 2 auf S. 8 aufgeführten Testfällen, soll mit einer sinnvollen Testabdeckung durch JUnit-Tests eine hohe Softwarequalität und -robustheit erreicht werden.

Nr.	Testfall	Beschreibung	Erwartetes Ergebnis
1	Consistent **Repository**	Das **Repository** wird über definierte Schnittstellen zu verschiedenen Laufzeiten angesprochen.	Daten sind vorhanden und können nur vom **Repository** modifiziert werden.
2	Task statistics validated	Es werden *Structured Query Language (SQL)*-Abfragen bei verschiedenen Datenbankständen durchgeführt.	Korrekte Ergebnisse der Tasks-Statistik werden geliefert.
3	Connectivity	Beim Aufrufen von *VMS* wird das Netzwerk-Traffic überwacht.	Client-Server Verbindung besteht (XMLHttpRequest auf "pending").
4	Incoming event	Es wird ein Event publiziert.	Im Frontend wird die JavaScript-Methode `onMessage` angesprochen.
5	Data transfer	Es wird ein Event mit Inhalt publiziert.	Event-Daten sind im *JSON*-Format.
6	Frontend event handling	Es wird ein Event publiziert.	Task-Event sorgt für Aktualisierung der *UI*-Task-Status-Anzeige.
7	User scoping	Es werden Events mit unterschiedlichen Zielgruppen/-personen publiziert.	Nur gewünschte Benutzer werden benachrichtigt.
8	Accessibility	Die Weboberfläche wird S/W betrachtet.	Status-Anzeigen sind barrierefrei Nutzbar.
9	Browser compatibility	Die Weboberfläche wird mit den vier "major browser"[9] getestet.	Das Design funktioniert bei allen angemessen.

Tabelle 2: Testfälle

5 Realisierungsphase

Zur Dokumentierung dieser Phase wird eine Art „Bottom-up-Prinzip“ genutzt. Dazu werden technisch oder fachlich getrennte Teilbereiche näher beleuchtet und dann in Zusammenhang gebracht.

5.1 Projektstruktur

Es wird die im VMS verwendete Paketstruktur übernommen. Diese ist angelehnt an das „Domain Driven Design“[10] aufgebaut. Die Struktur startet unterhalb von „com.<mandant>.contrax“, dabei steht <mandant> für das Kürzel eines Kunden. Auf der obersten Ebene befinden sich die fachlichen Säulen, gefolgt von den technischen Layern. Eine Erklärung dieser ist in einer tabellarischen und auszugsweisen Form im Anhang A.2.2 auf S. vi zu finden. Durch die Implementierung des Benachrichtigungssystems wurden die in Abbildung 3 gestrichelten Packete hinzugefügt. Der Großteil der neuen Dateien konnte aber in die bisherige Struktur eingegliedert werden.

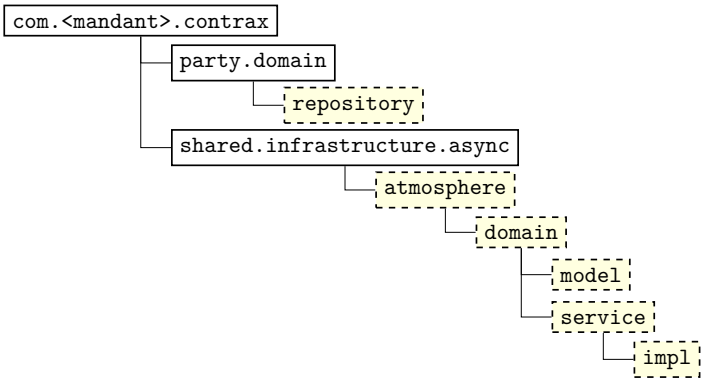


Abbildung 3: Differenz zur vorherigen Packetstruktur

5.2 Einrichten/Konfigurieren von Atmosphere

Das Framework wird wie im Anhang A.3.7: pom.xml (Auszug) auf S. xiii gezeigt per Maven[11] in der Version 2.2.6 eingebunden. In *VMS* werden alle Servlet-Container Komponenten (wie Filter und Listener) in der `VmsWebAppInitializer` konfiguriert. Hier wird die code-basierte Alternative, einer Servlet 3.0+ Umgebung, zu einer klassischen web.xml-Konfiguration genutzt. Nachdem das Atmosphere-Servlet im Webapplikationskontext unter dem Pfad `"/notify/*"` registriert wurde, wird es über eine **Spring**-annotierte Konfigurationsklasse (siehe Anhang A.3.2: AtmosphereConfig.java auf S. viii) entsprechend eingerichtet. Für eine asynchrone Verarbeitung muss das Atmosphere-Servlet sowie alle unter diesem Pfad wirksamen Filter und Listener mit dem Attribut „asyncSupported=true“ konfiguriert werden. (siehe Anhang A.3.13: VmsWebAppInitializer.java (Auszug) auf S. xx).

5.3 Kommunikation: Server zu Client

In diesem Abschnitt wird hauptsächlich der Code vom Anhang A.3.5: NotificationServiceHandler.java auf S. x erläutert und sich auf Anhang A.3.3: main.js (Auszug) auf S. viii bezogen. Außerdem werden unter Anderem einige Java-Klassen aus dem Framework Atmosphere namentlich genannt und daher im Folgenden erklärt. Für einen besseren Lesefluss wird bei den Begriffen der Präfix „Atmosphere“ im weiteren Verlauf vernachlässigt.

AtmosphereHandler entspricht einem Servlet. Jeder `AtmosphereHandler` ist einem konkretem Pfad unterhalb von `"/notify/*"` zugeordnet und übernimmt im Allgemeinen die Handhabung von POST/GET-Requests.

[9] Internet Explorer (bis Version 7 pflicht), Mozilla Firefox, Google Chrome, Opera.
[10] Definition siehe [DDD 2009].
[11] Maven ist ein „Build-Management-Tool“ basierend auf Java.

AtmosphereResource ist eine Java(-Wrapper)-Klasse die einen HTTP-Request, also ein Verbindung, darstellt. Über die jeweilige Instanz kann eine Verbindung z. B. angehalten und Informationen über diese abgerufen werden.

AtmosphereSession ermöglicht im Backend die persistente Speicherung von Informationen über eine Verbindung. Jede `AtmosphereResource` kann eine `AtmosphereSession` haben, welche auch nur solange lebt, wie die `AtmosphereResource` selbst.

Broadcaster sind Nachrichtenverteiler. Über die letztlich „Events“ mittels der Methode `broadcast` übermittelt werden können. Jede `AtmosphereResource` Instanz wird einem `Broadcaster` zugeteilt, welcher dann z. B. über `AtmosphereResource#getBroadcaster()` erreichbar ist. Zusätzlich kann eine `AtmosphereResource` mehrere `Broadcaster` abonnieren. Nur die abonnierten `Broadcaster` wissen dann von der `AtmosphereResource`, nicht andersrum. Über eine entsprechende sog. Factory[12] können `Broadcaster` ermittelt werden.

5.3.1 Verbindungshaltung

Bei jedem Seitenwechsel stellt der Client per JavaScript eine zusätzliche Anfrage (XMLHttpRequest) über den `"/notify/*"` Pfad an den Server. Diese **Resource** wird vom entsprechenden **Handler** jedoch nicht beantwortet, sondern bis auf weiteres „suspended“. Die Verbindung wird daher aufrecht erhalten und ermöglicht dem Server im späteren Verlauf den Client/Browser bei Bedarf zu benachrichtigen. Wird ein Event veröffentlicht, werden die entsprechenden **Resource**s beantwortet. Unmittelbar nach Erhalt dieser Antwort, sendet der Client erneut eine Anfrage. Dieses Vorgehen wurde bereits als „long-polling“ im Kapitel 3.2: Soll-Konzept auf Seite 4 aufgezeigt.
Atmosphere stellt bei der Verwaltung dieser **Resource**s sicher, dass es nur eine **Resource** pro Browser gibt. Unter Anderem durch die Vergabe einer einmaligen Id für jeden Browser.

5.3.2 Identifikation einer Verbindung

Nachdem eine **Resource** suspendiert wurde, beschafft die `AtmosphereResourceSessionFactory` die zugehörige **Session**. Mittels **Spring** wird der Benutzer ermittelt und seine Id in der **Session** gespeichert. Im späteren Verlauf kann so eine **Resource** dem jeweiligen Benutzer zugeordnet werden. Dies ist nötig, wenn z. B. explizit nur ein bestimmter Benutzer benachrichtigt werden soll.

Soll eine Benutzergruppe über ein Event informiert werden, so wird zur Identifikation der Verbindungen eine Reihe von **Broadcaster**n genutzt. Bei der Initialisierung vom `NotificationServiceHandler` wird dazu eine Map[13] von **Broadcaster**n pro Benutzergruppe instanziiert. Anschließend wird eine Liste von allen dem System bekannten Benutzergruppen erstellt. Zuletzt wird für jeden Eintrag ein **Broadcaster**, mit dem Gruppennamen als Kennung, definiert und der Map userRoleBroadcasterMap hinzugefügt. Nun kann der Nachrichtenverteiler für eine Benutzergruppe mittels der `BroadcasterFactory` und der Kennung ermittelt werden.

5.4 Datenhaltung

An dieser Stelle wird aufgezeigt, wie die Webanwendung bestimmte Informationen abspeichert. Es wird auf viele Anhänge aus dem Kapitel A.3: Quellcode auf S. vii und zu Teilen aus dem Kapitel A.1: Weboberfläche auf S. ii Bezug genommen.

[12] Eine Factory kapselt die Logik zur Erstellung eines Objektes von der eigentlichen Business-Logik ab. An die Factory-Methode übergebene Daten werden dann zur Instanziierung eines bestimmten Objekttyps genutzt, meist ist dies eine abgeleitete Klasse z. B. eines Interfaces (zu Deutsch: Schnittstelle). [vgl. AVAJAVA.COM 2015]
[13] Liste von Schlüssel/Wert-Paaren.

5.4.1 UserStatusRepository

Repository Ein Repository (zu Deutsch: Ablage, Aufbewahrungsort) stellt ein Mittel dar, die Geschäftslogik von der Beschaffung und/oder der Instanziierung zu trennen. Erzeugte Objekte liegen dann im Repository abrufbereit. Häufig genutzte Lese-, Schreib- und Änderungsvorgänge werden oftmals direkt vom Repository zur Verfügung gestellt. [vgl. Norbert Eder 2011]

Zum Zeitpunkt der Implementierung wird bisher nur der letzte Besuch der Offenen-Task-Übersicht eines jeden Benutzers im **Repository** gespeichert. Die Ermittlung des Zeitstempels wird unten stehend erklärt. Da es sich lediglich um ein DateTime-Objekt handelt, könnte diese Information in der vorhandenen Datenbank gespeichert werden. Gemäß dem Abschnitt Datenhaltung aus Kapitel 4.1: Technisches Konzept auf Seite 7 wurde jedoch das **Repository** entwickelt.

5.4.1.1 Letzter Besuch der Offenen-Task-Übersicht Beim Aufruf dieser Übersicht wird zunächst der aktuelle „zuletzt besucht"-Zeitstempel beim `UserStatusRepository` erfragt. Existiert für den genannten Benutzer kein Zeitstempel, so wird der Zeitpunkt „01.01.1970 00:00:00"[14] angelegt. Der Zeitstempel wird dann für die Markierung neuer Tasks in der Weboberfläche genutzt.
Da ein Aufruf der genannten Seite gleichzeitig diesen Zeitstempel verändern soll, musste dem JSF-Lebenszyklus[15] ein „PostRenderView"-Event hinzugefügt werden. Dieses Event wird erst ausgelöst, wenn eine Seite fertig gebaut ist und somit der bisherige Zeitstempel von der Weboberfläche verwendet wurde. Wurde das Event auf der besagten Seite (listOpenTasks.xhtml) ausgelöst wird die Methode `OpenTaskListController#visitedOpenTasks()` aufgerufen, die den Prozess zur Aktualisierung des Zeitstempels anstößt.

5.4.2 TaskNotificationStatisticsModel

Diese Java-Klasse hält die Daten bereit, die für die Darstellung der Status-Anzeige[16] benötigt werden und wird von der Weboberfläche[17] über den `NotificationController` genutzt. Dieses „Model" ist ein Ergebnis der *SQL*-Abfrage in `getTaskNotificationModel(VmsUser user, DateTime lastVisit)` der Klasse `TaskQueryService`. Dabei werden Informationen über die Task-Typen Fehler und Benutzerinteraktion abgefragt. Dazu gehört die Anzahl offener Tasks eines Typs und ob jeweils neue Tasks seit dem letzten Besuch der Offenen-Tasks-Übersicht hinzugekommen sind. Jede Information wird in einer Spalte angelegt. Anhand der Spaltenbezeichnung wird die Information entsprechend in das `TaskNotificationStatisticsModel`-Objekt zugeordnet. Dabei handelt es sich entweder um einen numerischen oder booleschen Wert.

5.5 Datenübertragung

Bei derzeitiger Implementierung, nach Kapitel 4.1: Technisches Konzept auf Seite 7, muss jede Event-Klasse die Klasse `AtmosphereEvent` erweitern, und kann ggf. die Methode `toJSON()` überschreiben. Eine Beispielimplementierung stellt dabei die Klasse `TaskNotificationEvent` dar. In dieser wird ersichtlich wie Daten pro Task-Typ abgelegt werden können. Es wurde darauf verzichtet, die `AtmosphereEvent#toJSON()` anzupassen. Dieses Event informiert den Client lediglich über neue Tasks, enthält aber nicht die zur Anzeige benötigten Informationen (Anzahl der offenen Task pro Typ). Diese werden wie in Kapitel Kapitel 5.6: Event-Handling auf Seite 12 beschrieben anschließend vom Client abgefragt. Die direkte Übertragung der Informationen hat für Probleme gesorgt. Zur Einhaltung des Zeitrahmens wurde der o.g. Lösungsansatz gewählt.

[14] Datum-Bedeutung: http://openbook.rheinwerk-verlag.de/javainsel9/javainsel_15_002.htm (Aufruf: 26.03.15).
[15] Erklärung: http://jsfatwork.irian.at/book_de/jsf.html#!idx:/jsf.html:2.6 (Aufruf: 26.03.15).
[16] Siehe Kapitel 3.2: Soll-Konzept auf Seite 4.
[17] In diesem Kontext von Anhang A.1.4: masterTemplate.xhtml (Auszug) auf S. v.

5.6 Event-Handling

In diesem Kapitel wird aufgezeigt, wie Events im Backend publiziert werden können und wie diese im Frontend behandelt werden. Es wird Bezug auf Anhang A.3.10: TaskNotificationService.java auf S. xv, Anhang A.3.3: main.js (Auszug) auf S. viii, Anhang A.3.4: NotificationController.java auf S. ix und Anhang A.1.3: main.less (Auszug) auf S. iii genommen.

5.6.1 Im Backend

Im Backend wird ein Service (`TaskNotificationService`) bereitgestellt, welcher zur Publizierung von Task-Benachrichtigungs-Events über **Atmosphere** genutzt wird.
Ein konkreter Service zum Versenden von Events implementiert die im Interface `NotificationService` deklarierte Methode `pushNotification`. Diese Methode ist mehrfach überladen. Je nach Ziel kann der Methode als Parameter einen Benutzer oder eine Benutzergruppe übergeben werden. Bei Ersterem reicht die Benutzer-Id. Es gibt eine `AbstractNotificationService` Klasse mit wichtigen Hilfsmethoden die von jedem derartigen Service benötigt werden.

5.6.2 Im Frontend

Beim Client wird das Event-Handling über den `onMessage`-Listener geregelt. Siehe Anhang A.3.3: main.js (Auszug) auf S. viii. Alle Events, die vom Server veröffentlicht werden, kommen bei erfolgreicher Übermittlung bei dieser Methode an. Zunächst wird versucht, den Nachrichteninhalt in ein gültiges *JSON*-Objekt umzuwandeln. Falls die Konvertierung scheitert, wird bei bisheriger Implementierung die Nachricht vom Server nicht neu versendet. War die Umwandlung erfolgreich, so wird zur Unterscheidung der Event-Typen die Variable `event` mit einem entsprechenden String verglichen. Dazu wird Kenntnis über die serverseitige Enum-Definition dieser Strings benötigt.
Liegt ein Task-Benachrichtigungs-Event vor, wird mittels **jQuery** ein Klick auf einem versteckten Button in der Weboberfläche ausgelöst. Dadurch wird der Prozess zur *UI*-Aktualisierung angestoßen. Genauer wird zunächst das dahinterliegende Model der Status-Anzeigen zurückgesetzt, anschließend wird ein „partial Ajax-Request“ ausgelöst. Dieser fordert das genannte Model beim Server an, da es zurückgesetzt wurde, wird es von diesem neu gebaut. Zum Schluss wird das nun aktualisierte Model zum Rendern der neuen Status-Anzeige genutzt.

5.7 UI-Entwurfsumsetzung

Die spezifischen und konkreten Ideen aus Kapitel 3.2 auf Seite 4 und Kapitel 4.2 auf Seite 7 wurden umgesetzt. Der Task-Typ Benutzerinteraktion erscheint in einem blauen Oval, während die Status-Anzeige der Error-Tasks im roten Rechteck angezeigt wird. Durch die Formen allein sind die Task-Typen bereits unterscheidbar. Zusätzlich besitzt jede Status-Anzeige einen sog. Tooltip[18], welcher kurz und schlüssig die Anzeigen differenziert und erklärt. Dadurch kann auf eine Anwenderdokumentation verzichtet werden.

In der Task-Übersicht ist erkennbar, dass außerdem alle weiteren Task-Typen einem Pastellton zugeordnet wurden. Darin ist ebenfalls der Hover-Effekt[19] sichtbar, der zur Verschönerung der Status-Anzeigen und des Farbbalkens hinzugefügt wurde. Angelehnt am bereits vorhandenem Hover-Effekt der Tabelle, bei der eine Zeile, über der die Maus liegt, subtil farblich hervorgehoben wird. So wird auch der farbliche Balken und die Status-Anzeige in der Farbtiefe angehoben (Siehe farblicher Balken in Anhang A.1.1: UI-Entwurf auf S. ii).

[18] Wenn die Maus über eine Element gehalten wird, erscheint eine kleine Infobox neben der Maus.
[19] Wenn die Maus über eine Element gehalten wird, wird dieses Element gesondert hervorgehoben.

6 Qualitätssicherungsphase

Um, nach Abschluss der Realisierungsphase, die Vollständigkeit der Umsetzung gegenüber den umzusetzenden Anforderungen und die Funktionstüchtigkeit der Erweiterung sicherzustellen, werden zahlreiche Tests durchgeführt.

6.1 Definition of Done

Das Definition of Done (DoD) ist eine Art Vereinbarung, die definiert wann eine Aufgabe, wie die Erweiterung, als fertig anzusehen ist. In diesem Fall wurde innerhalb des Entwicklerteams eine solche Vereinbarung getroffen, dabei steht die Sicherstellung der Funktionalität und die Wahrung der Softwarequalität im Vordergrund. Beim Sichten des Programmcodes wird nicht nur das eigene Verständnis dessen geschärft, sondern auch verbessert und schlanker gemacht. Frei nach dem Prinzip des Pfadfinders, wie es im Rotem Grad der Clean Code Developer Initiative[20] beschrieben ist:

„Hinterlasse einen Ort immer in einem besseren Zustand als du ihn vorgefunden hast."

Da zeitgleich komplexere Abschnitte des Programmes kommentiert werden, wird dieser auch verständlicher. Durch die sinnvollen Kommentare, inklusive JavaDoc, kann auf eine externe Entwicklerdokumentation verzichtet werden.

6.2 Testergebnisse

Das Resultat der ausgeführten Tests wird in der Tabelle 3: Testergebnisse festgehalten. In Tabelle 2: Testfälle auf S. 8 sind die durchgeführten Tests beschrieben.

Nr.	Testfall	Auswertung
1	Consistent**Repository**	✓
2	Task statistics validated	✓
3	Connectivity	✓
4	Incomingevent	✓
5	Data transfer	✓
6	Frontend event handling	✓
7	User scoping	✓
8	Accessibility	✓
9	Browsercompatibility	✓

Tabelle 3: Testergebnisse

7 Abschlussphase

Zum Ende der Dokumentation wird das Projekt in der Retroperspektive betrachtet. In einem Soll-Ist-Vergleich wird geprüft, ob alle Anforderungen beachtet und die Zeitplanung eingehalten wurde. Abschließend wird das Projekt mit einem persönlichen Fazit bewertet.

7.1 Soll-Ist-Vergleich

Alle Anforderungen an die Erweiterung konnten erfüllt werden. Die Art der Aktualisierung der Status-Anzeigen ist, nach meinem Empfinden, unzufrieden gelöst. Siehe Kapitel 5.5 auf Seite 11. Dies musste jedoch zur Einhaltung des zeitlichen Rahmens vorerst hingenommen werden.
In Tabelle 4 auf S. 14 werden der angenommene und der tatsächliche benötigte Zeitaufwand verglichen. Wie man erkennen kann, dauerte die Durchführung des Proof-of-Concept wesentlich länger als geplant. Dies lag daran, dass das Framework **Atmosphere** nicht in einem Umfang wie gewünscht dokumentiert ist. Auch da es stetig Änderungen am Framework gibt, sind Demo-Programmcodes häufig veraltet gewesen. Der Mehraufwand reduzierte jedoch die benötigte Zeit für die Einarbeitung ins Framework und das Erstellen eines Grundgerüstes. Der restliche Mehraufwand verkleinerte den Puffer für Anforderungsänderungen. Es ist zu sehen, dass dieser gänzlich ausgeschöpft wurde. Zunächst für seinen geplanten Sinn, in der markierten Zeile, für die Umsetzung einer Anforderungsänderung.

[20] Vgl. http://www.clean-code-developer.de/Roter-Grad.ashx#Praktiken_10 (Aufruf: 02.04.2015).

Der übrige Puffer konnte dann für eine bessere *UI*-Umsetzung und eine umfangreichere Qualitätssicherung genutzt werden.

Vorgangsname	Dauer	
	Geplant	Änderung
Analysephase	**6,5 Std.**	
Mögliche zukünftige Anforderungen abschätzen	1,5 Std.	
Mögliche Frameworks evaluieren	4 Std.	
Kosten-Nutzen-Analyse: Make-or-Buy Framework	1 Std.	
Konzeptphase	**13,5 Std.**	**+ 4 Std.**
Technisches Konzept definieren	6 Std.	
UI-Entwurf erstellen	3 Std.	
Proof-of-Concept durchführen	3 Std.	+ 4 Std.
Testplan aufstellen	1,5 Std.	
Realisierungphase	**34 Std.**	**- 6 Std.**
Framework: Einarbeiten und Grundgerüst herstellen	8 Std.	- 4 Std.
Konzept für Verbindungshaltung anwenden	4,5 Std.	
Benutzer-Status-Repository anlegen	2 Std.	+ 1 Std.
Service zur Publizierung von Events bereitstellen	3 Std.	
Eventhandling im Backend implementieren	2,5 Std.	
UI-Entwurf umsetzen	6 Std.	+ 1 Std.
Eventhandling in JavaScript implementieren	4 Std.	
Puffer für Anforderungsänderungen	4 Std.	- 4 Std.
Testphase	**3,5 Std.**	**+ 2 Std.**
Definition of Done	2 Std.	+ 2 Std.
Tests absolvieren	1,5 Std.	
Abschlussphase	**12,5 Std.**	
Soll-Ist-Vergleich	1,5 Std.	
Projektdokumentation erstellen	11 Std.	
Summe	**70 Std.**	

Tabelle 4: Ursprüungliche Zeitplanung und tatsächlich benötigte Zeit im Vergleich

Erklärung der unterstrichenen Zeile: Die aus dem Projektantrag stammende Bezeichnung „Datenbank und Entität anpassen“ ist aufgrund einer Anforderungsänderung nicht mehr zutreffend und wurde angepasst.

7.2 Persönliches Fazit

Das Projektthema, die Realisierung eines Benachrichtigungssystems, hat mich seit Nennung im *VMS*-Projekt als Anforderung gereizt. Es ist nicht nur ein schönes IHK-Abschlussprojekt, es ist auch ein sehr eigenständiges und abgrenzbares Feature. Ich konnte mich dabei fachlich weiterbilden, da ich mich in meinem Projektthema nicht auf bereits Beherrschtes beschränkt habe. Darüber hinaus war eine persönliche Entwicklung zum besseren Anwendungsentwickler zu sehen. Ich freue mich auch künftig am Projekt *VMS* mitwirken zu können, sowie das Thema der Erweiterung weiter zu begleiten.

Literaturverzeichnis

DDD 2009
Erklärung des domain-driven Design. http://dddsample.sourceforge.net/architecture.html. Version: März 2009, Abruf: 26.03.2015

avajava.com 2015
Code Strategies: *Erklärung des Factory Pattern.* http://www.avajava.com/tutorials/lessons/factory-pattern.html, Abruf: 26.03.2015

Björn Nowak 2015
Björn Nowak: *IHK Abschlussprojekt-Antrag.* Januar 2015

BTC 2014
BTC AG: *Unternehmensprofil.* https://www.btc-ag.com/de/unternehmen.htm, Abruf: 19.03.2015

BTC 2015
BTC AG: *Produktseite von TSO Business Service Suit.* https://www.btc-ag.com/de/Energie-TSO-Business-Service-Suite.htm, Abruf: 19.03.2015

ITwissen.info 2015
DATACOM Buchverlag GmbH: *Erklärung von jQuery.* http://www.itwissen.info/definition/lexikon/jQuery.html, Abruf: 23.03.2015

Joe Hanson 2014
Joe Hanson: *Was ist HTTP Long Polling?* http://www.pubnub.com/blog/http-long-polling/. Version: Dezember 2014, Abruf: 04.05.2015

Norbert Eder 2011
Norbert Eder: *Erklärung vom Repository Pattern.* https://www.norberteder.com/das-repository-pattern-anhand-eines-beispiels-inkl-tests. Version: Juli 2011, Abruf: 30.03.2015

SELFHTML e.V. 2015
SELFHTML e.V.: *Erklärung von JavaScript Object Notation (JSON).* http://wiki.selfhtml.org/wiki/JavaScript/JSON, Abruf: 30.03.2015

UWS Ltd 2015
UWS Ltd: *Was ist Java EE / Java Enterprise?* http://uws-software-service.com/de/leistungen/java-kompetenz/softwareentwicklung-mit-dem-java-enterprise-framework.html, Abruf: 23.03.2015

Wikipedia 2015
Wikimedia Foundation Inc.: *Erklärung von Spring.* http://de.wikipedia.org/wiki/Spring_%28Framework%29, Abruf: 23.03.2015

A Anhang

A.1 Weboberfläche

A.1.1 UI-Entwurf

Zu sehen ist die Übersichtsseite der offenen Tasks. Neue Tasks werden fett-gedruckt dargestellt. Die Zeile auf der sich die Maus befindet wird bereits hervorgehoben. Daher soll sich dies auch bei dem farbigen Balken, links jeden Eintrages, wiederspiegeln. (Siehe sechster Eintrag von oben)

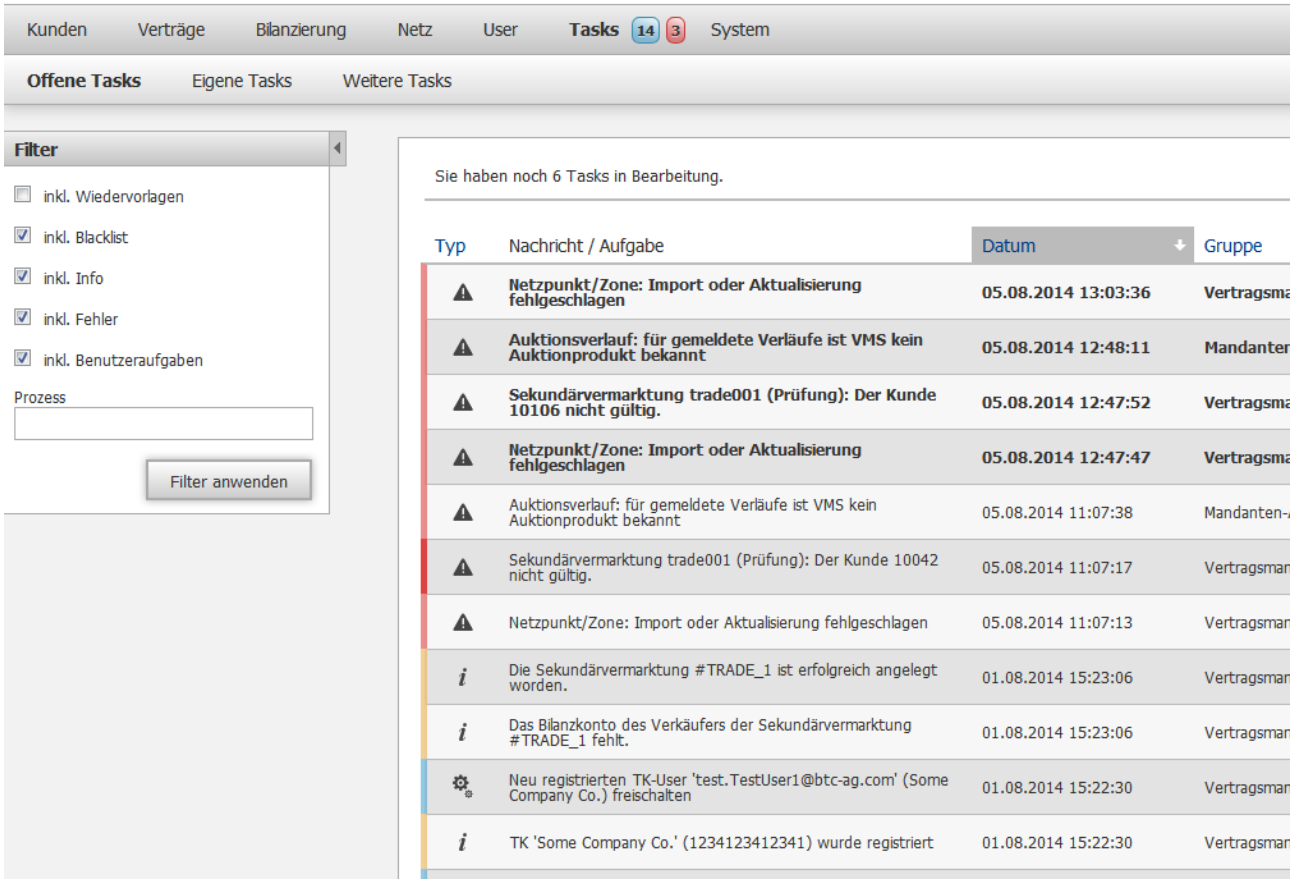


Abbildung 4: UI-Entwurf

A.1.2 listOpenTasks.xhtml (Auszug)

```
1 <ui:define name="metadata">
2     <f:metadata>
3         <f:event type="postRenderView" listener="#{openTaskList.visitedOpenTasks()}"/>
4     </f:metadata>
5 </ui:define>
```

A.1.3 main.less (Auszug)

```
1  .taskNotification {
2    @default-base: lightgrey;
3    @default-hover: darkgrey;
4    @default-border: #909090;
5
6    @error-base: rgba(222, 67, 67, 0.60);
7    @error-hover: #DE4343;
8    @error-border: #892d2d;
9
10   @user_action-base: rgba(78, 165, 205, 0.60);
11   @user_action-hover: #4EA5CD;
12   @user_action-border: #28526b;
13
14   @info-base: rgba(234, 175, 81, 0.60);
15   @info-hover: #eaaf51;
16   @info-border: #b3802d;
17
18   @resubmission-base: rgba(97, 184, 50, 0.60);
19   @resubmission-hover: #61B832;
20   @resubmission-border: #204312;
21
22
23   // style for table design: left color bar (for all task tables)
24   & td:first-of-type {
25     border-left: 6px solid @default-base;
26
27     &:hover, &.hover {
28       border-left-color: @default-hover;
29     }
30     &.error {
31       border-left-color: @error-base;
32       &:hover, &.hover {
33         border-left-color: @error-hover;
34       }
35     }
36     &.user_action {
37       border-left-color: @user_action-base;
38       &:hover, &.hover {
39         border-left-color: @user_action-hover;
40       }
41     }
42     &.info {
43       border-left-color: @info-base;
44       &:hover, &.hover {
45         border-left-color: @info-hover;
46       }
47     }
48     &.resubmission {
49       border-left-color: @resubmission-base;
50       &:hover, &.hover {
51         border-left-color: @resubmission-hover;
52       }
53     }
54   }
55
56   // style for task notification container in menu
57   & a {
58     -webkit-border-radius: 10px;
59     -moz-border-radius: 10px;
```

```
60      border-radius: 6px;
61      .font(14px);
62      color: #323232;
63      padding: 2px 3px;
64      margin-left: 5px;
65      text-shadow: 0 1px 0 rgba(255, 255, 255, .5);
66
67      // default color style
68      border: solid 1px @default-border;
69      .linear-gradient(200deg, white, 0%, @default-base, 80%, right top, left bottom);
70      &:hover, &.hover {
71        .linear-gradient(200deg, white, 0%, @default-hover, 80%, right top, left bottom);
72      }
73
74      &.error {
75        -webkit-border-radius: 0;
76        -moz-border-radius: 0;
77        border-radius: 0;
78        background: 0;
79
80        height: 20px;
81        display: inline-block;
82        margin-left: 5px;
83        padding: 0 5px;
84
85        border: 1px solid @error-border;
86        .linear-gradient(200deg, white, 0%, @error-base, 80%, right top, left bottom);
87        &:hover, &.hover {
88          .linear-gradient(200deg, white, 0%, @error-hover, 80%, right top, left bottom);
89        }
90      }
91
92      &.user_action {
93        border-radius: 90px;
94        text-align: center;
95        height: 20px;
96        display: inline-block;
97        margin: 0 2px 0 8px;
98        padding: 0px 8px;
99
100       border: 1px solid @user_action-border;
101       .linear-gradient(200deg, white, 0%, @user_action-base, 80%, right top, left bottom);
102       &:hover, &.hover {
103         .linear-gradient(200deg, white, 0%, @user_action-hover, 80%, right top, left bottom);
104       }
105     }
106
107     &.info {
108       border-color: @info-border;
109       .linear-gradient(200deg, white, 0%, @info-base, 80%, right top, left bottom);
110       &:hover, &.hover {
111         .linear-gradient(200deg, white, 0%, @info-hover, 80%, right top, left bottom);
112       }
113     }
114
115     &.resubmission {
116       border-color: @resubmission-border;
117       .linear-gradient(200deg, white, 0%, @resubmission-base, 80%, right top, left bottom);
118       &:hover, &.hover {
119         .linear-gradient(200deg, white, 0%, @resubmission-hover, 80%, right top, left bottom);
120       }
```

```
121     }
122   }
123 }
```

A.1.4 masterTemplate.xhtml (Auszug)

```
1  <!-- ... -->
2  <c:forEach items="#{menuController.menu}" var="menu">
3      <c:if test="#{menu.rendered}">
4          <li>
5              <h:commandLink action="#{menu.navigate()}"
6                             value="#{menu.label}"
7                             styleClass="#{menu.isActive() ? 'active' : ''} #{menu.label eq 'Tasks' ? '
                                  taskNotification' : ''}"
8                             rendered="#{vms:isAllowed(menu.view.viewId, null)}"/>
9
10             <c:if test="#{menu.label eq 'Tasks'}">
11                 <h:panelGroup id="taskNotification" styleClass="taskNotification">
12                     <h:commandLink styleClass="user_action"
13                                    action="#{openTaskList.showOnlyActionTasks()}"
14                                    value="#{notifications.taskStatistic.userActionTaskCount}"
15                                    style="#{notifications.taskStatistic.hasNewUserActionTasks ? 'font-weight:
                                          bold' : ''}"
16                                    title="#{globalBundle['taskNotification_userAction']}"/>
17                     <h:commandLink styleClass="error"
18                                    action="#{openTaskList.showOnlyErrorTasks()}"
19                                    value="#{notifications.taskStatistic.errorTaskCount}"
20                                    style="#{notifications.taskStatistic.hasNewErrorTasks ? 'font-weight: bold' : ''}
                                          "
21                                    title="#{globalBundle['taskNotification_error']}"
22                                    rendered="#{notifications.taskStatistic.errorTaskCount > 0}"/>
23                 </h:panelGroup>
24                 <h:commandButton id="taskNotification-update-btn" style="display: none"
25                                  action="#{notifications.resetTaskTypesStatistic()}">
26                     <f:ajax render="taskNotification"/>
27                 </h:commandButton>
28             </c:if>
29
30             <!-- submenu rendering -->
31         </li>
32     </c:if>
33 </c:forEach>
34 <!-- ... -->
```

A.2 Tabellen

A.2.1 Laptop-Spezifikation

Herrsteller/Modell	HP ProBook 6460b
CPU	Intel® Core™ i5-2520M (2,50 GHz, 3MB L3-Cache)
RAM	8 GB, 1333 MHz, DDR3 SDRAM
Grafikkarte	Intel® HD Graphics 3000
Festplatte	320 GB, 7200 U/min, SATA II
OS	Windows 7

Tabelle 5: Laptop-Spezifikation

A.2.2 Auszug der vorhandene Paketstruktur

Ein vollständiges Beispielpacket wäre: com.<mandant>.contrax.contract.domain.model.transactions

Package	Beschreibung
contract	**Verträge mit Transaktionen**
party	**Mandanten, Kunden und Personen**
bg	**Bilanzkreise mit Ein-/Ausbringungen**
grid	**Gasnetz**
shared	
task	

Tabelle 6: Packetstruktur: Fachliche Saeulen

Package	Beschreibung
view	**Klassen für das Benutzerinterface**
application	**Klassen für die Programmsteuerung**
domain	**Domänenmodell und -Services und -Prozesse**
domain.model	Klassen des Datenmodells
domain.query	Queryservices (Interfaces)
domain.service	Services der Domäne (Aufruf insb. durch: View-Layer und Prozesse)
domain.process	Prozesse der Domäne (verwenden Services)
infrastructure	**Klassen für die Infrastruktur der Anwendung**
infrastructure.query	Implementierungen der Queryservices.
infrastructure.ws	Webservices

Tabelle 7: Packetstruktur: Technische Layer

A.3 Quellcode

Bei Java-Dateien wurden zur Übersichtlichkeit sämtliche Importierungen mit „import *;“ abgekürzt. Ebent so sämtliche Kommentare, einschließlich JavaDoc, sofern diese nicht maßgeblich zum Verständnis beitragen. Es wird alphabetisch nach Dateinamen aufgelistet.

A.3.1 AbstractNotificationService.java

```
1  package com.oge.contrax.shared.infrastructure.async.atmosphere.domain.service.impl;
2
3  import *;
4
5  public abstract class AbstractNotificationService implements NotificationService {
6
7      @InjectLogger
8      protected Logger logger;
9
10     @Inject
11     protected transient UserQueryService userQueryService;
12
13     // TODO @Inject with Atmosphere v2.3, when released
14     protected AtmosphereFramework atmosphere() {
15         return (AtmosphereFramework) ServletContextFactory.getDefault()
16                 .getServletContext().getAttribute("atmosphereServlet");
17     }
18
19     protected BroadcasterFactory broadcasterFactory() {
20         return atmosphere().getBroadcasterFactory();
21     }
22
23     protected AtmosphereResourceSessionFactory sessionFactory() {
24         return AtmosphereResourceSessionFactory.getDefault();
25     }
26     // end of to-do
27
28
29     /**
30      * publish event for specified broadcaster
31      * @param b
32      * @param event
33      */
34     protected void broadcast(Broadcaster b, AtmosphereEvent event) {
35         try {
36             b.broadcast(event.toJSON());
37         } catch (IOException e) {
38             logger.error("Could not map to JSON", e);
39         }
40     }
41
42     /**
43      * publish event only for given resource using the specified broadcaster\n
44      * TODO broadcaster may not necessary - cause default broadcaster is stored in resource
45      * @param b
46      * @param event
47      * @param resource
48      */
49     protected void broadcast(Broadcaster b, AtmosphereEvent event, AtmosphereResource resource) {
50         try {
51             b.broadcast(event.toJSON(), resource);
52         } catch (IOException e) {
```

```
53                 logger.error("Could not map to JSON", e);
54             }
55         }
56 }
```

A.3.2 AtmosphereConfig.java

```
1  package com.oge.contrax.shared.infrastructure.config;
2
3  import *;
4
5  @Configuration
6  public class AtmosphereConfig {
7
8      public static void init(ServletRegistration.Dynamic atmosphereRegistration) {
9          atmosphereRegistration.setAsyncSupported(true);
10         atmosphereRegistration.setInitParameter(
11                 "org.atmosphere.cpr.packages", "com.oge.contrax.shared.infrastructure.async.atmosphere");
12         atmosphereRegistration.setInitParameter(
13                 "org.atmosphere.interceptor.HeartbeatInterceptor.paddingChar", "hb");
14         atmosphereRegistration.setInitParameter(
15                 "org.atmosphere.interceptor.HeartbeatInterceptor.heartbeatFrequencyInSeconds", "300");
16         atmosphereRegistration.setInitParameter(
17                 "org.atmosphere.interceptor.HeartbeatInterceptor.clientHeartbeatFrequencyInSeconds", "0");
18         atmosphereRegistration.setInitParameter(
19                 "org.atmosphere.cpr.broadcaster.shareableThreadPool", "true");
20     }
21 }
```

A.3.3 main.js (Auszug)

Die Methode, welche den Hover-Effekt in der Task-Übersicht ermöglicht, und die Methode `configureListeners(request,socket)` wurden aufgrund der Anhangsbegrenzung ausgelassen. In Letzterer werden sog. Listener, wie `onOpen` und `onTransportFailure` definiert. Der `onMessage`-Listener wurde wegen seiner Wichtigkeit gesondert implementiert.

```
1  function subscribeForNotification(scope) {
2      if (vms.isDevelopment) {
3          console.log("subscribeForNotification");
4      }
5
6      var $navFormScope = $('#navForm');
7
8      // configuration of connection
9      var api = atmosphere;
10     var request = new api.AtmosphereRequest();
11     request.url = 'http://' + document.location.host + '/notify';
12     request.transport = "long-polling";
13     request.fallbackTransport = "streaming";
14     request.contentType = 'application/json';
15     request.shared = true;
16     request.reconnectInterval = 1000 * 10;
17     request.trackMessageLength = true;
18     request.logLevel = (vms.isDevelopment ? 'debug' : 'info');
19
20
21     /**
22      * Everytime a new message is received
```

```
23      * @param response should be a json of type tasksData or other implemented event types
24      */
25     request.onMessage = function (response) {
26         if (vms.isDevelopment) {
27             console.log("onMessage");
28         }
29 
30         var message = response.responseBody;
31         try {
32             var json = $.parseJSON(message);
33         } catch (e) {
34             // TODO may request re-send of event?
35             console.log('Not a valid JSON: ', message);
36             return;
37         }
38         console.log(json);
39 
40 
41         // event control for task notifications
42         if (json.tasksData != undefined) {
43             // for non-data event
44             $("#navForm\\:notification-update-btn", $navFormScope).click();
45 
46             /*
47             // for data event
48             var $boxUserAction = $(".notification > a.user_action", $navFormScope);
49             var $boxError = $(".notification > a.error", $navFormScope);
50             json.tasksData.forEach(function (task) {
51                 switch (task.type) {
52                     case "USER_ACTION":
53                         $boxUserAction.text(task.count);
54                         break;
55                     case "ERROR":
56                         $boxError.text(task.count);
57                         break;
58                 }
59             });
60             */
61         }
62 
63     };
64 
65     var socket = null;
66     configureListeners(request, socket);
67 
68     // connect to server with defined parameters
69     socket = api.subscribe(request);
70 
71 }
```

A.3.4 NotificationController.java

```
1 package com.oge.contrax.party.view.user;
2 
3 import *;
4 
5 @Controller("notifications")
6 @Scope("session")
7 public class NotificationController implements Serializable {
```

```
8
9       @InjectLogger
10      private Logger logger;
11
12      @Inject
13      private transient TaskQueryService taskQueryService;
14
15      @Inject
16      private transient UserStatusRepository userStatusRepository;
17
18      private TaskNotificationStatisticsModel model;
19
20
21      public void resetTaskTypesStatistic() {
22          model = null;
23      }
24
25      public TaskNotificationStatisticsModel getTaskStatistic() {
26          if (model == null) {
27              updateModel();
28          }
29          return model;
30      }
31
32      /**
33       * will be called by OpenTaskListController, when ever a update is needed
34       * @see com.oge.contrax.task.view.OpenTaskListController#visitedOpenTasks()
35       */
36      public void updateModel() {
37          model = taskQueryService.getTaskNotificationModel(
38                  SecurityUtils.getCurrentUser(),
39                  userStatusRepository.getLastOpenTaskVisit(SecurityUtils.getCurrentUserId()));
40      }
41  }
```

A.3.5 NotificationServiceHandler.java

```
1   package com.oge.contrax.shared.infrastructure.async.atmosphere;
2
3   import *;
4
5   /**
6    * general notification service endpoint \n
7    * 13.03.15 : no need of different connection handling for different events
8    */
9   @AtmosphereHandlerService(
10          path = "/notify",
11          supportSession = true,
12          broadcasterCache = UUIDBroadcasterCache.class,
13          interceptors = {
14                  AtmosphereResourceLifecycleInterceptor.class,
15                  AtmosphereResourceStateRecovery.class,
16                  OnDisconnectInterceptor.class,
17                  SessionCreationInterceptor.class,
18                  TrackMessageSizeInterceptor.class,
19                  WebSocketMessageSuspendInterceptor.class,
20                  IdleResourceInterceptor.class,
21                  HeartbeatInterceptor.class,
22                  JavaScriptProtocol.class})
23  public class NotificationServiceHandler
```

```
24          extends AbstractReflectorAtmosphereHandler
25          implements AtmosphereHandler, AtmosphereServletProcessor {
26
27      private transient Logger logger = LoggerFactory.getLogger(NotificationServiceHandler.class);
28      private transient BroadcasterFactory broadcasterFactory;
29      private transient AtmosphereResourceSessionFactory sessionFactory;
30      private Map<UserRole, Broadcaster> userRoleBroadcasterMap;
31
32      @Override
33      public void init(AtmosphereConfig config) throws ServletException {
34          broadcasterFactory = config.getBroadcasterFactory();
35          sessionFactory = AtmosphereResourceSessionFactory.getDefault();
36
37          // init a list of broadcaster per any existing user role
38          userRoleBroadcasterMap = new HashMap<>();
39          List<UserRole> allRoles = UserRole.ALL_TENANT_USERS.asList();
40          allRoles.addAll(UserRole.ALL_SYSTEM_USERS.asList());
41          allRoles.addAll(UserRole.ALL_CUSTOMER_USERS.asList());
42          for (UserRole role : allRoles) {
43              Broadcaster b = lookupBroadcaster(role.getKey());
44              userRoleBroadcasterMap.put(role, b);
45          }
46      }
47
48      private Broadcaster lookupBroadcaster(String pathInfo) {
49          String[] decodedPath = pathInfo.split("/");
50          Broadcaster b = broadcasterFactory.lookup(decodedPath[decodedPath.length - 1], true);
51          return b;
52      }
53
54      @Override
55      public void destroy() {
56      }
57
58      /**
59       * is called for every request of service endpoint ("/notify")
60       * @param resource represent a http connection wrapped by atmosphere
61       * @throws IOException
62       */
63      @Override
64      public void onRequest(AtmosphereResource resource) throws IOException {
65          AtmosphereRequest request = resource.getRequest();
66          AtmosphereResponse response = resource.getResponse();
67          String method = request.getMethod();
68          VmsUser user = SecurityUtils.getCurrentUser();
69
70          if ("GET".equalsIgnoreCase(method)) {
71              resource.addEventListener(new WebSocketEventListenerAdapter());
72              response.setContentType("application/json");
73
74              // after this the actual connection will be suspended and receive a heartbeat per interval
75              resource.suspend();
76
77              // every resource have a session - lives as long a connection is alive
78              // and stores user id for later identification of an resource
79              AtmosphereResourceSession resourceSession = sessionFactory.getSession(resource);
80              resourceSession.setAttribute("vmsUserId", SecurityUtils.getCurrentUserId());
81
82              for (UserRole role : SecurityUtils.getCurrentUserRoles()) {
83                  Broadcaster b = userRoleBroadcasterMap.get(role);
84                  if (b != null) {
```

```
85                      b.addAtmosphereResource(resource);
86                  }
87              }
88
89              logger.trace("User {} with browser {} requested {}",
90                      new String[]{user.getUsername(), resource.uuid(), request.getPathInfo()});
91          } else if ("POST".equalsIgnoreCase(method)) {
92              logger.debug("(No action done) Browser {} has send '{}'", resource.uuid(), request.getReader().readLine
                    ().trim());
93          }
94      }
95
96      // if control of connection handling is needed - working example implementation
97      /*
98          ...
99      */
100 }
```

A.3.6 OpenTaskListController.java

```
1  package com.oge.contrax.task.view;
2
3  import *;
4
5  @Controller("openTaskList")
6  @Scope("view")
7  public class OpenTaskListController extends AbstractTaskListController {
8
9      private static final long serialVersionUID = -5685323931682143251L;
10
11     @Inject
12     private transient OpenTaskFilter filter;
13
14     @Inject
15     private transient UserStatusRepository userStatusRepository;
16
17     @Inject
18     private transient NotificationController  notificationController ;
19
20     @PostConstruct
21     @Override
22     protected void init() {
23         super.init();
24     }
25
26     public String showOnlyActionTasks() {
27         OpenTaskFilter newFilter = new OpenTaskFilter(SecurityUtils.getCurrentUser(), SecurityUtils.
                getCurrentUserRoles());
28         newFilter.setIncludeUserAction(true);
29         newFilter.setIncludeError(false);
30         newFilter.setIncludeInfo(false);
31         newFilter.setIncludeBlacklisted(false);
32         newFilter.setIncludeResubmissions(false);
33          filter . setPriorFilter (newFilter);
34         return View.TASK_LIST_OPEN_VIEW.redirect().outcome();
35     }
36
37     public String showOnlyErrorTasks() {
38         OpenTaskFilter newFilter = new OpenTaskFilter(SecurityUtils.getCurrentUser(), SecurityUtils.
                getCurrentUserRoles());
```

```
39          newFilter.setIncludeUserAction(false);
40          newFilter.setIncludeError(true);
41          newFilter.setIncludeInfo(false);
42          newFilter.setIncludeBlacklisted(false);
43          newFilter.setIncludeResubmissions(false);
44           filter.setPriorFilter(newFilter);
45          return View.TASK_LIST_OPEN_VIEW.redirect().outcome();
46      }
47
48      public void visitedOpenTasks() {
49          DateTime visitedAt = TimeUtils.now();
50          userStatusRepository.updateLastOpenTaskVisit(SecurityUtils.getCurrentUserId(), visitedAt);
51           notificationController.updateModel();
52
53          logger.debug("User " + SecurityUtils.getCurrentUsername() + "(" + SecurityUtils.getCurrentUserId()
54                  + ") visited open tasks list at " + visitedAt);
55      }
56
57      public DateTime getLastVisitOpenTask() {
58          // exclude requests by ajax i.e.
59          if (FacesContext.getCurrentInstance().isPostback()) {
60              return null;
61          }
62          return userStatusRepository.getLastOpenTaskVisit(SecurityUtils.getCurrentUserId());
63      }
64
65      @Override
66      public TaskFilter getFilter() {
67          return filter.get();
68      }
69  }
```

A.3.7 pom.xml (Auszug)

```
1  <!-- ... -->
2  <properties>
3    <!-- ... -->
4    <atmosphere.version>2.2.6</atmosphere.version>
5  </properties>
6
7  <dependencies>
8    <!-- ... -->
9    <dependency>
10     <groupId>org.atmosphere</groupId>
11     <artifactId>atmosphere-runtime</artifactId>
12     <version>${atmosphere.version}</version>
13   </dependency>
14 </dependencies>
15 <!-- ... -->
```

A.3.8 PostRenderView-Event

Zum leztlichen Eintragen des Events in den JSF-Lebenszyklus, muss der sog. „Phaselistener“ als dieser in die faces-config.xml eingetragen werden.

```
1  package com.oge.contrax.shared.infrastructure.jsf;
2
3  import *;
4
5  @NamedEvent(shortName="postRenderView")
6  public class PostRenderViewEvent extends ComponentSystemEvent {
7
8      public PostRenderViewEvent(UIComponent component) {
9          super(component);
10     }
11
12 }
```

```
1  package com.oge.contrax.shared.infrastructure.jsf;
2
3  import *;
4
5  public class PostRenderViewListener implements PhaseListener {
6
7      @Override
8      public PhaseId getPhaseId() {
9          return PhaseId.RENDER_RESPONSE;
10     }
11
12     @Override
13     public void beforePhase(PhaseEvent event) {
14     }
15
16     @Override
17     public void afterPhase(PhaseEvent event) {
18         FacesContext context = FacesContext.getCurrentInstance();
19         context.getApplication().publishEvent(context, PostRenderViewEvent.class, context.getViewRoot());
20     }
21
22 }
```

A.3.9 TaskNotificationEvent.java

```
1  package com.oge.contrax.shared.infrastructure.async.atmosphere.domain.model;
2
3  import *;
4
5  /**
6   * represent a event published to the client \n
7   * 12.03.15 : only used for identification of event type by client -
8   *            actually no data will be send therefore deprecate-marked methods
9   */
10 public class TaskNotificationEvent extends AtmosphereEvent {
11
12     public enum TaskType {ERROR, USER_ACTION, INFO, BLACKLIST, RESUBMISSION}
13
14     private List<TaskTypeData> tasksData;
15
16     public TaskNotificationEvent() {{
```

```
17          event = EventType.TASK_NOTIFY;
18          tasksData = new ArrayList<>();
19      }
20
21      @Deprecated
22      public TaskNotificationEvent addData(TaskType taskType, int taskCount) {
23          tasksData.add(new TaskTypeData(taskType, taskCount));
24          return this;
25      }
26
27      @Deprecated
28      public List<TaskTypeData> getTasksData() {
29          return tasksData;
30      }
31
32      public class TaskTypeData {
33          private TaskType type;
34          private Integer count;
35
36          public TaskTypeData(TaskType type, Integer count) {
37              this.type = type;
38              this.count = count;
39          }
40
41          public TaskType getType() {
42              return type;
43          }
44
45          public Integer getCount() {
46              return count;
47          }
48      }
49  }
```

A.3.10 TaskNotificationService.java

```
1   package com.oge.contrax.shared.infrastructure.async.atmosphere.domain.service.impl;
2
3   import *;
4
5   /**
6    * service to publish task events
7    */
8   @Service
9   public class TaskNotificationService extends AbstractNotificationService {
10      private final String mainBroadcasterPath = "/notify";
11
12      @Override
13      public void pushNotification(VmsUserId targetUserId) {
14          Broadcaster b = broadcasterFactory().lookup(mainBroadcasterPath, false);
15          if (b != null) {
16              for (AtmosphereResource resource : b.getAtmosphereResources()) {
17                  AtmosphereResourceSession resourceSession = sessionFactory().getSession(resource, false);
18                  if (resourceSession != null) {
19                      VmsUserId resourceUserId = resourceSession.getAttribute("vmsUserId", VmsUserId.class);
20                      if (resourceUserId.equals(targetUserId)) {
21                          broadcast(b, new TaskNotificationEvent(), resource);
22                          return;
23                      }
24                  }
```

```
25                  logger.debug("There is no session for {}", targetUserId);
26              }
27          } else {
28              logger.debug("There is no broadcaster for \"" + mainBroadcasterPath + "\"");
29          }
30      }
31
32      @Override
33      public void pushNotification(VmsUser targetUser) {
34          pushNotification(targetUser.getUserId());
35      }
36
37      @Override
38      public void pushNotification(UserRole role) {
39          Broadcaster b = broadcasterFactory().lookup(role.getKey(), false);
40          if (b != null) {
41              broadcast(b, new TaskNotificationEvent());
42          } else {
43              logger.debug("There is no broadcaster for role {}", role.getKey());
44          }
45      }
46
47      // example push services for event including data
48      /*
49          ...
50      */
51  }
```

A.3.11 TaskQueryServiceImpl.java (Auszug)

Interface:

```
1  package com.oge.contrax.task.domain.query;
2
3  import *;
4
5  public interface TaskQueryService extends QueryService<Task, TaskNumber> {
6
7      // ...
8
9      /**
10      * count tasks per specified types for given user and checks if there are new tasks since given lastVisit
11      * @param user filter for specific user
12      * @return the statistical figures for the ui task notification
13      */
14     TaskNotificationStatisticsModel getTaskNotificationModel(VmsUser user, DateTime lastVisit);
15
16     // ...
17 }
```

Implementierung:

```
1  package com.oge.contrax.task.domain.query.impl;
2
3  import *;
4
5  @Service
6  @Transactional(readOnly = true, value = TenantDataConfig.TX_MANAGER_NAME)
7  public class TaskQueryServiceImpl extends QueryServiceSupportJpa<Task, TaskNumber> implements
       TaskQueryService {
```

```
8
9       // ...
10
11      /**
12       * {@inheritDoc}
13       */
14      @Override
15      public TaskNotificationStatisticsModel getTaskNotificationModel(VmsUser user, DateTime lastVisit) {
16          TaskNotificationStatisticsModel model = new TaskNotificationStatisticsModel();
17
18          if (user != null) {
19              Query query = em().createNativeQuery(
20                      "SELECT 'errorTaskCount', COUNT(t.PK) FROM Task t "
21                          + " WHERE (t.OWNERGROUP_PK IN (:roles) "
22                              + " OR t.EDITOR_UID = :user) "
23                              + " AND t.closedAt IS NULL "
24                              + " AND t.category = 0 "
25                      + " UNION "
26                      + " SELECT 'userActionTaskCount', COUNT(t.PK) FROM Task t "
27                          + " WHERE (t.OWNERGROUP_PK IN (:roles) "
28                              + " OR t.EDITOR_UID = :user) "
29                              + " AND t.closedAt IS NULL "
30                              + " AND t.category = 1 "
31                      + " UNION "
32                      + " SELECT 'hasNewUserActionTask', COUNT(t.PK) FROM Task t "
33                          + " WHERE (t.OWNERGROUP_PK IN (:roles) "
34                              + " OR t.EDITOR_UID = :user) "
35                              + " AND t.closedAt IS NULL "
36                              + " AND t.category = 1 "
37                              + " AND t.created_At > :lastVisit "
38                      + " UNION "
39                      + " SELECT 'hasNewErrorTask', COUNT(t.PK) FROM Task t "
40                          + " WHERE (t.OWNERGROUP_PK IN (:roles) "
41                              + " OR t.EDITOR_UID = :user) "
42                              + " AND t.closedAt IS NULL "
43                              + " AND t.category = 0 "
44                              + " AND t.created_At > :lastVisit ");
45
46              List<String> roles = new ArrayList<>();
47              for (UserRole userRole : user.getRoles()) {
48                  roles.add(userRole.name());
49              }
50              query.setParameter("user", user.getUserId().getValue());
51              query.setParameter("roles", roles);
52              query.setParameter("lastVisit", new Timestamp(lastVisit.getMillis()));
53
54              List<Object[]> result = (List<Object[]>) query.getResultList();
55              for (Object[] entry : result) {
56                  String type = (String) entry[0];
57                  int count = ((BigDecimal) entry[1]).intValue();
58                  boolean value = count > 0;
59
60                  switch (type) {
61                      case "errorTaskCount":
62                          model.setErrorTaskCount(count);
63                          break;
64                      case "userActionTaskCount":
65                          model.setUserActionTaskCount(count);
66                          break;
67                      case "hasNewUserActionTask":
68                          model.setHasNewUserActionTasks(value);
```

```
69                          break;
70                      case "hasNewErrorTask":
71                          model.setHasNewErrorTasks(value);
72                          break;
73                  }
74              }
75          }
76          return model;
77      }
78
79      // ...
80  }
```

A.3.12 UserStatus(Repository).java

```
1  package com.oge.contrax.party.domain.model.user;
2
3  import *;
4
5  public class UserStatus {
6
7      private VmsUserId userId;
8
9      private DateTime lastOpenTaskVisit;
10
11     public UserStatus(VmsUserId userId) {
12         this.userId = userId;
13     }
14
15     public UserStatus(UserStatus copyOf) {
16         this.userId = copyOf.userId;
17         this.lastOpenTaskVisit = copyOf.lastOpenTaskVisit;
18     }
19
20     public DateTime getLastOpenTaskVisit() {
21         return lastOpenTaskVisit;
22     }
23
24     /**
25      * should only be used by repository, else changes wont affect in repository
26      * @param lastOpenTaskVisit
27      * @see com.oge.contrax.party.domain.repository.UserStatusRepository#updateLastOpenTaskVisit
28      */
29     public void setLastOpenTaskVisit(DateTime lastOpenTaskVisit) {
30         this.lastOpenTaskVisit = lastOpenTaskVisit;
31     }
32
33     /**
34      * used by repository
35      * @see com.oge.contrax.party.domain.repository.UserStatusRepository#clone
36      */
37     public UserStatus clone() {
38         return new UserStatus(this);
39     }
40 }
```

```
1  package com.oge.contrax.party.domain.repository;
2
3  import *;
4
```

```
5  @Singleton
6  @Component
7  public class UserStatusRepository {
8
9      private Map<VmsUserId, UserStatus> repository;
10
11     public UserStatusRepository() {
12         this.repository = new HashMap<>();
13     }
14
15     public UserStatus get(VmsUserId vmsUserId) {
16         UserStatus userStatus = fetch(vmsUserId);
17         return clone(userStatus);
18     }
19
20     public UserStatus remove(VmsUserId vmsUserId) {
21         UserStatus userStatus = repository.remove(vmsUserId);
22         return clone(userStatus);
23     }
24
25     private UserStatus store(VmsUserId vmsUserId) {
26         repository.put(vmsUserId, new UserStatus(vmsUserId));
27         return get(vmsUserId);
28     }
29
30     private UserStatus fetch(VmsUserId vmsUserId) {
31         UserStatus userStatus = repository.get(vmsUserId);
32         if (userStatus == null) {
33             userStatus = store(vmsUserId);
34         }
35         return userStatus;
36     }
37
38     private UserStatus clone(UserStatus userStatus) {
39         UserStatus clone = null;
40         if (userStatus != null) {
41             clone = userStatus.clone();
42         }
43         return clone;
44     }
45
46     // UserStatus getter and setter
47
48     public void updateLastOpenTaskVisit(VmsUserId vmsUserId, DateTime visit) {
49         UserStatus userStatus = fetch(vmsUserId);
50         userStatus.setLastOpenTaskVisit(visit);
51     }
52
53     public DateTime getLastOpenTaskVisit(VmsUserId vmsUserId) {
54         DateTime lastOpenTaskVisit = get(vmsUserId).getLastOpenTaskVisit();
55         if (lastOpenTaskVisit == null) {
56             lastOpenTaskVisit = TimeUtils.BEGIN_OF_TIME;
57         }
58         return lastOpenTaskVisit;
59     }
60
61 }
```

A.3.13 VmsWebAppInitializer.java (Auszug)

```
1  package com.oge.contrax.shared.infrastructure.config;
2
3  import *;
4
5  /**
6   * Code-based alternative to web.xml for use within Servlet 3.0+ environments. See
7   * {@link WebApplicationInitializer} Javadoc for complete details.
8   */
9  public class VmsWebAppInitializer implements WebApplicationInitializer {
10
11     private static final Logger LOG = LoggerFactory.getLogger(VmsWebAppInitializer.class);
12     public static final EnumSet<DispatcherType> ALL_DISPATCHERS = EnumSet.allOf(DispatcherType.class);
13
14
15     /**
16      * Register and configure all Servlet container components necessary to power the
17      * Contrax web application.
18      */
19     @Override
20     public void onStartup(ServletContext sc) throws ServletException {
21         LOG.debug("{}.onStartup()", getClass().getName());
22
23         // Create the 'root' Spring application context
24         AnnotationConfigWebApplicationContext context = new AnnotationConfigWebApplicationContext();
25         context.setServletContext(sc);
26
27         // ...
28
29         Dynamic atmosphereRegistration = registerServlet(sc, AtmosphereServlet.class, 2, "/notify/*");
30         AtmosphereConfig.init(atmosphereRegistration);
31     }
32
33     private FilterRegistration.Dynamic addFilter(ServletContext sc, String filterName, Filter filter ) {
34         FilterRegistration.Dynamic dynamic = sc.addFilter(filterName, filter);
35         dynamic.setAsyncSupported(true);
36         return dynamic;
37     }
38
39     private FilterRegistration.Dynamic addFilter(ServletContext sc, String filterName, Class<? extends Filter>
           filterClass) {
40         FilterRegistration.Dynamic dynamic = sc.addFilter(filterName, filterClass);
41         dynamic.setAsyncSupported(true);
42         return dynamic;
43     }
44
45     // ...
46 }
```

A.4 Weitere Grafiken

Alle Grafiken sind in ihrer Größe reduziert. In den Klassendiagrammen werden ausschließlich Klassen betrachtet die hinzugefügt oder inhaltlich geändert wurden. Vererbungen und Implementierungen sind, sofern von Relevanz, angegeben.

A.4.1 UML-Klassendiagramm (Übersicht)



Abbildung 5: UML-Klassendiagramm (Übersicht)

A.4.2 UML-Klassendiagramm (Komplex)

Eine Blau-geschriebene Methode weist darauf hin, dass diese verändert wurde. Eine in Grün geschrieben, dass diese neu hinzugefügt wurde. Beinhaltet eine dargestellte Klasse nur Letzteres ist davon auszugehen, dass diese gänzlich Neu ist.

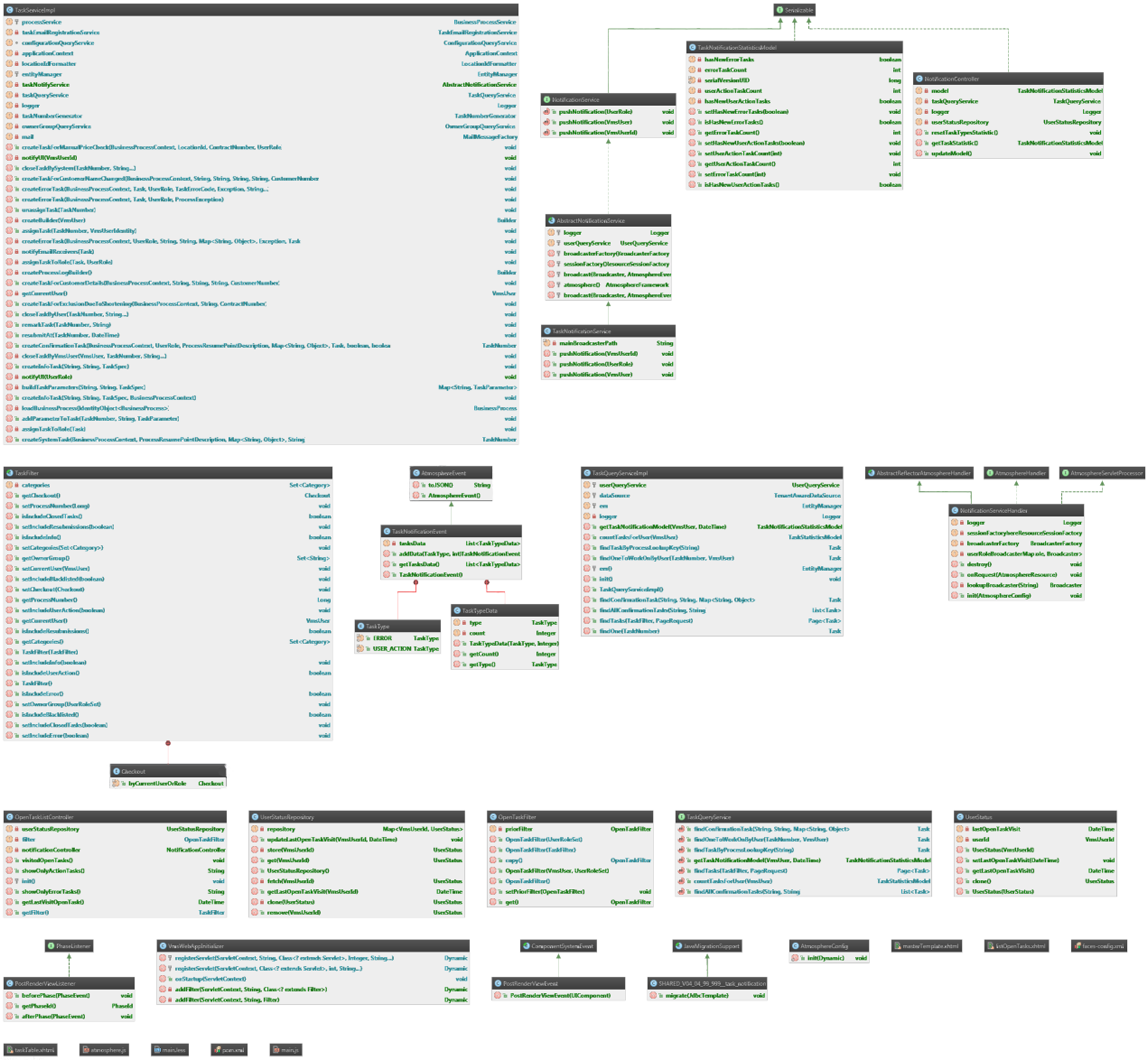


Abbildung 6: UML-Klassendiagramm (Komplex)